Software-Defined Networking

Chapitre 3

Le protocole Openflow

M. BOUHORMA

DÉPARTEMENT G. INFORMATIQUE, FSTT, UAE





Plan du cours sur OpenFlow: Objectifs pédagogiques :

- Comprendre le **principe d'OpenFlow** et son rôle dans les réseaux SDN.
- Configurer et manipuler une **topologie Mininet**.
- Utiliser des **flux OpenFlow** pour contrôler le comportement des paquets.
- Interagir avec un **contrôleur OpenFlow** (ONOS, OpenDaylight ou Ryu).
- Analyser et dépanner les **flux réseau** en temps réel.

Objectif : Situer OpenFlow dans le contexte du SDN.

- Transition des réseaux traditionnels vers SDN.
- **Concept-clé :** Séparation du plan de contrôle (control plane) et du plan de données (data plane).

Exemple :

Avec les réseaux traditionnels, les équipements comme les switchs et routeurs gèrent à la fois la logique de contrôle et le traitement des paquets.

SDN permet de déléguer cette logique à un contrôleur centralisé, facilitant la gestion et l'automatisation.

OpenFlow est un des premiers et plus populaires protocoles permettant cette interaction.

1. **Introduction à OpenFlow**

2. **Concepts fondamentaux d'OpenFlow**
    1. Plan de contrôle vs Plan de données
    2. Fonctionnement d'un switch OpenFlow

3. **Table de flux et règles OpenFlow**
    1. Éléments d'une table de flux

4. **Messages OpenFlow**
    1. Types de messages

5. **Schéma du fonctionnement d'OpenFlow**



1. **Démonstration TP pratique avec Mininet et ONOS**
    1. Pré-requis
    2. Installation et configuration
    3. Inspection des messages avec Wireshark

2. **Étude de cas et ateliers pratiques**
3. **Bonnes pratiques et considérations avancées**
4. **Conclusion**
5. **Questions et discussion**

Sujets du cours :

- Introduction au SDN et à la softwarisation des réseaux
- Fondamentaux d'OpenFlow et des contrôleurs SDN
- Virtualisation de réseau et réseaux superposés
- Cas d'utilisation du SDN dans les réseaux de centres de données et le cloud computing
- Sécurité et gestion du SDN
- Tendances futures du SDN et de la softwarisation des réseaux

Travaux pratiques : Tout au long de ce cours, les étudiants travailleront sur différents travaux pratiques couvrant
Les concepts fondamentaux du SDN et de la softwarisation des réseaux, y compris le protocole OpenFlow, les contrôleurs SDN et la virtualisation de réseau.
De plus, les travaux pratiques se concentreront sur l'application du SDN et de la softwarisation des réseaux dans des scénarios réels tels que les réseaux de centres de données et le cloud computing.
En réalisant ces travaux pratiques, les étudiants acquerront des compétences pratiques en configuration de topologies réseau, commutateurs virtuels et routeurs, et en test de la performance du réseau.

TP & Ateliers

Introduction à Mininet et création d'une topologie réseau simple
Objectif : Simuler une topologie basique avec Mininet, comprenant des hôtes et des commutateurs, et tester la connectivité réseau.

Contrôle de flux réseau avec OpenFlow
Objectif : Configurer et gérer des flux réseau en utilisant OpenFlow et un contrôleur SDN (comme OpenDaylight), pour comprendre comment SDN sépare le plan de contrôle du plan de données.

Équilibrage de Charges Dynamique avec OpenFlow et OpenDaylight

Objectif : Mettre en place un mécanisme d'équilibrage de charge pour répartir dynamiquement le trafic entre plusieurs serveurs en fonction des règles définies par OpenDaylight.

Contrôle de Flux Dynamique en Temps Réel avec OpenFlow

Objectif : Modifier dynamiquement les règles de flux pour gérer le trafic en fonction des conditions du réseau (ex. surcharge, latence élevée).

Configuration Automatique des Commutateurs via OpenDaylightObjectif : Démontrer comment OpenDaylight peut automatiquement configurer des commutateurs dans un réseau SDN.

L’architecture traditionnelle



Qu'est-ce que le plan de contrôle ?

Le plan de contrôle est la partie d'un réseau qui contrôle la façon dont les paquets de données sont acheminés. Le processus du routage, par exemple, est considéré comme faisant partie du plan de contrôle.

Les routeurs utilisent divers protocoles pour identifier les chemins du réseau, et ils stockent ces chemins dans des tables de routage.

Qu'est-ce que le plan de données ? Qu'est-ce que le plan de transfert ?

Contrairement au plan de contrôle, qui détermine comment les paquets doivent être transmis, le plan de données transmet réellement les paquets.

Pensez au plan de contrôle comme étant les feux de signalisation qui fonctionnent aux intersections d'une ville. Le plan de données (ou plan d'acheminement), quant à lui, ressemble davantage aux voitures qui circulent sur les routes, s'arrêtent aux intersections et respectent les feux de signalisation.

Les avantages du SDN (Software-Defined Networking)

Les **avantages du SDN (Software-Defined Networking)** sont nombreux et touchent différents aspects des réseaux modernes, en particulier la flexibilité, l'efficacité, et la sécurité.
Voici une synthèse des principaux bénéfices :

1. Centralisation du contrôle du réseau

- Le SDN utilise un **contrôleur central** pour gérer tout le réseau de manière cohérente.
- Cela permet une vue globale du réseau, facilitant la gestion des configurations et réduisant les erreurs manuelles.

2. Flexibilité et Programmabilité

- Grâce à des interfaces programmables (comme OpenFlow), les administrateurs peuvent automatiser des tâches (ajout de règles de routage, gestion de la bande passante, etc.).
- Cela permet d'adapter dynamiquement les réseaux en fonction des besoins métiers ou des changements dans le trafic.

3. Amélioration de l'efficacité et réduction des coûts

- En séparant le plan de contrôle (contrôleur SDN) du plan de données (switches/routeurs), le SDN optimise l'utilisation des ressources du réseau.
- Il est possible d'utiliser des équipements réseau moins coûteux (switches « whitebox ») sans compromettre la performance, car l'intelligence est déportée dans le contrôleur.

Les avantages du SDN (Software-Defined Networking)

4. Facilité de déploiement et d'administration

- Le SDN permet une **configuration simplifiée** via des interfaces centralisées. Par exemple, une nouvelle politique de sécurité ou un service peut être déployé rapidement sur l'ensemble du réseau.
- L'intégration avec des **API** permet d'automatiser de nombreux processus, facilitant ainsi l'administration quotidienne.

5. Adaptabilité et gestion dynamique du trafic

- Le SDN permet de réagir en temps réel aux changements dans le réseau (comme les congestions) en redirigeant automatiquement le trafic vers des chemins optimisés.
- Cette gestion dynamique est particulièrement utile dans des environnements à forte charge, comme les centres de données ou les réseaux de télécommunication.

6. Meilleure sécurité réseau

- Le SDN permet de définir des règles de sécurité spécifiques de manière centralisée, et de les appliquer automatiquement sur l'ensemble du réseau.
- En cas d'attaque ou d'incident, le réseau peut être reconfiguré en temps réel pour isoler les parties vulnérables.

Les avantages du SDN (Software-Defined Networking)

7. Support pour la virtualisation et le cloud

- Le SDN s'intègre bien avec les environnements virtualisés (cloud) en facilitant le déploiement de **réseaux virtuels** ou d'infrastructures à la demande.
- Par exemple, des **VPC (Virtual Private Cloud)** peuvent être configurés rapidement avec des contrôles fins sur la sécurité et la connectivité.

8. Support des nouveaux paradigmes : SD-WAN et NFV

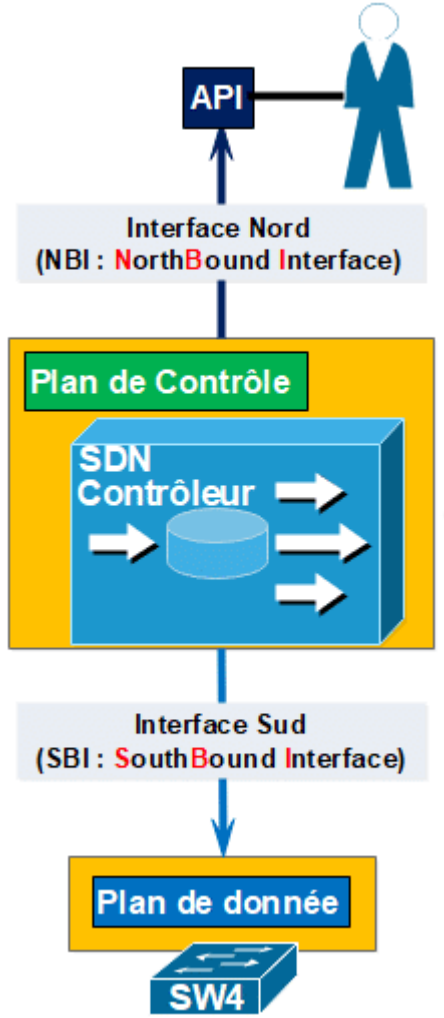
- Le SDN est un élément clé des architectures SD-WAN, qui permettent d'optimiser la connectivité entre différents sites distants.
- Il facilite également l'intégration de NFV (Network Functions Virtualization) en permettant l'orchestration et la gestion centralisée de fonctions réseau virtualisées (pare-feux, load balancers, etc.).

Conclusion

Le SDN est une solution puissante pour moderniser les infrastructures réseau en offrant une **meilleure gestion, plus de flexibilité et une sécurité accrue**. En automatisant et en centralisant le contrôle, il permet aux entreprises de répondre plus rapidement aux besoins évolutifs du marché, tout en réduisant les coûts opérationnels.

SDN: Fonctionnement

L'un des avantages d'un contrôleur central est que l'on peut configurer l'ensemble du réseau à partir d'un seul appareil.
Le contrôleur central a un **accès complet** et un aperçu de tout ce qui se passe dans le **réseau**.



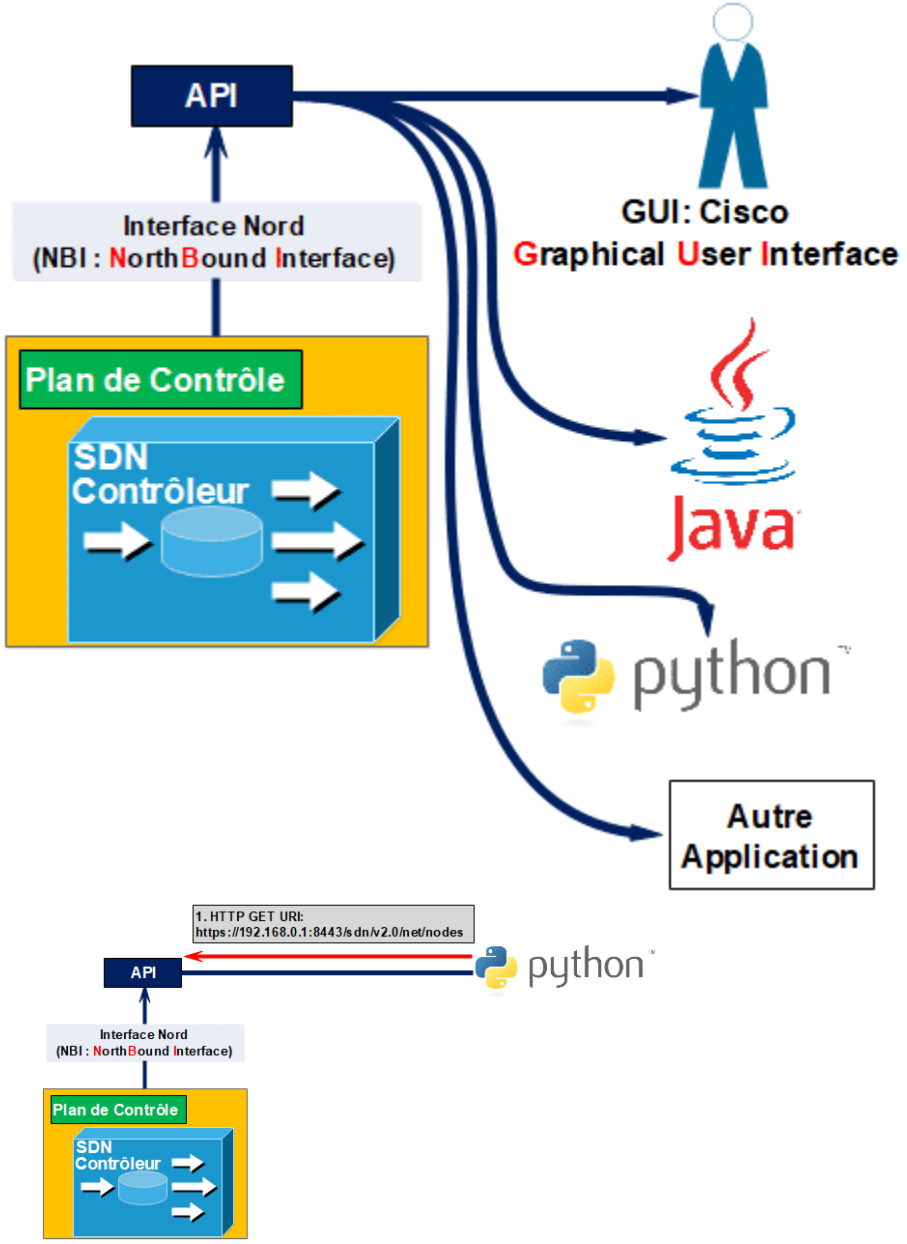
Le contrôleur SDN doit communiquer avec les périphériques réseau dans le but, de **programmer le plan de données.**

SDN: Fonctionnement

Une **interface graphique** (GUI), il y a aussi une **API** qui permet à d'autres applications d'accéder au contrôleur SDN.

On peut utiliser **l'API** pour écrire **des scripts et automatiser l'administration du réseau.** Voici quelques exemples de script, **qu'il est possible de lancer :**

- **On peut répertorier** les informations de tous les équipements de son réseau.
- **On peut afficher** l'état de toutes les interfaces physiques du réseau.
- **Ajoutez** un nouveau VLAN sur tous les switchs.
- **Affichez** la topologie de l'ensemble du réseau.
- **Et même, configurez** automatiquement les adresses IP, le routage et les ACL lorsqu'une nouvelle machine virtuelle est créée.



Software Defined Networking : Interface vers le sud (SBI : SouthBound Interface)

L'API est une **interface logicielle** qui permet à une application de donner accès à d'autres applications en utilisant **des fonctions et des structures de données prédéfinies**

3 types d'interfaces sud, les plus populaires :

- **L'OpenFlow** : est probablement le **SBI** (**S**outh**B**ound **I**nterface) le plus populaire en ce moment, c'est un protocole **open source de l'ONF** (**O**pen **N**etworking **F**oundation). **L'OpenFlow** est pris en charge par la plupart des périphériques réseau et des contrôleurs SDN.

- **Ensuite, on a le Cisco OpFlex** : qui est **la réponse de Cisco à l'OpenFlow.** C'est aussi un protocole open source qui a été soumis à **l'IETF** (**I**nternet **E**ngineering **T**ask **F**orce) pour le normaliser.

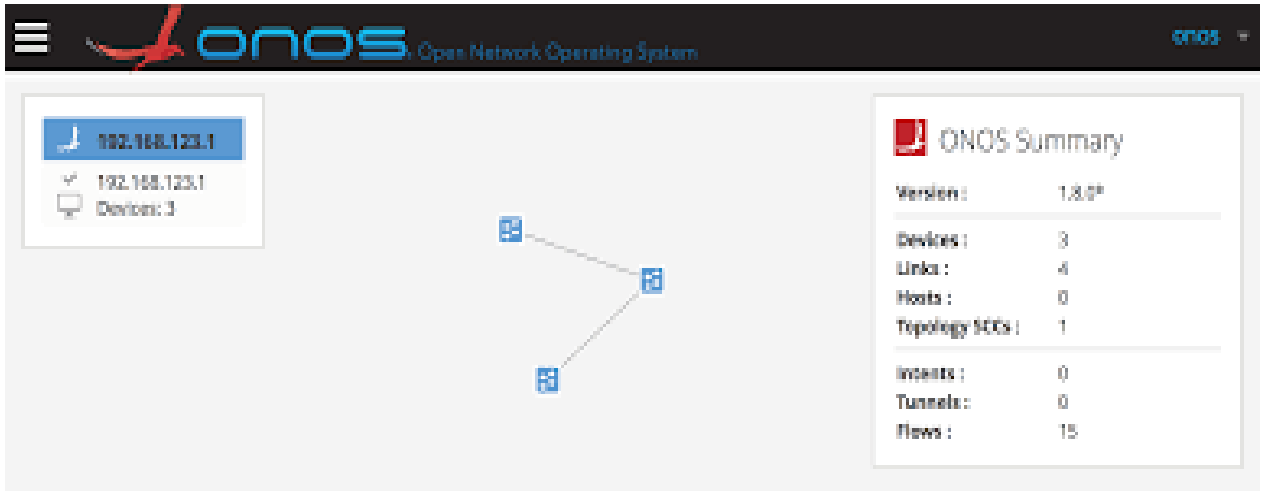
- **Et on a la traditionnelle CLI (Command-Line Interface)** : Cisco propose le **contrôleur APIC-EM** (**A**pplication **P**olicy **I**nfrastructure **C**ontroller **E**nterprise **M**odule) qui est une **solution SDN** ou solution Software Defined Networking.

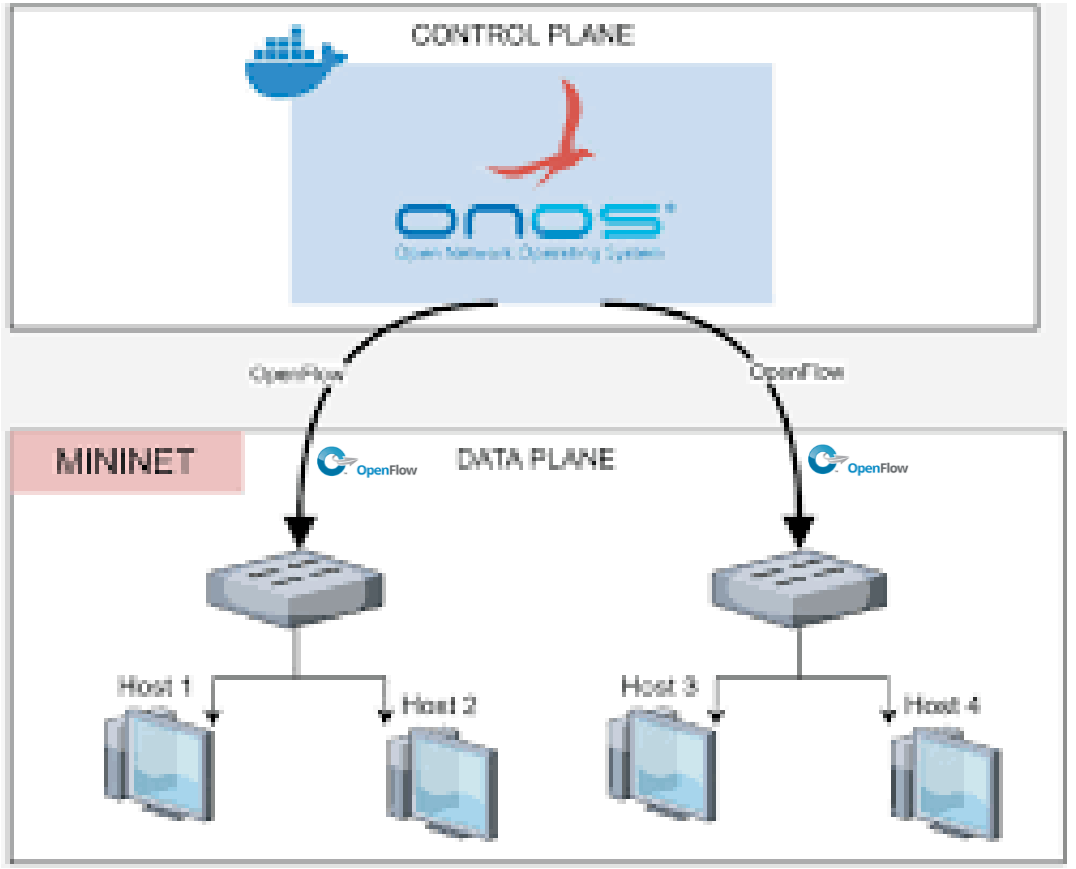
OpenFlow



APIC-EM
Version 1.3

OpenFlow Switch : Objectifs et mécanismes opérationnels





Plan d'application (Application Plane)

- Bien que ONOS appartienne principalement au plan de contrôle, il permet aux développeurs d'écrire **des applications SDN** qui peuvent fonctionner au-dessus de lui.
- Ces applications sont des modules additionnels (comme pour l'équilibrage de charge ou la surveillance) qui utilisent les API d'ONOS pour interagir avec le réseau.

Plan de contrôle (Control Plane): Rôle de ONOS

ONOS est un **contrôleur SDN** en gérant la logique de contrôle et en prenant des décisions sur la manière dont les paquets doivent circuler dans le réseau.

- Il maintient une vue globale et centralisée de la topologie réseau et distribue des règles de flux (flows) aux commutateurs OpenFlow.
- Le contrôleur ONOS est responsable de l'application des politiques de routage, de l'équilibrage de charge, et de la gestion des chemins.

Au cœur de la technologie SDN se trouve le protocole OpenFlow, et le SDN avec un commutateur OpenFlow promet une flexibilité et une configuration rapide des réseaux de communication.

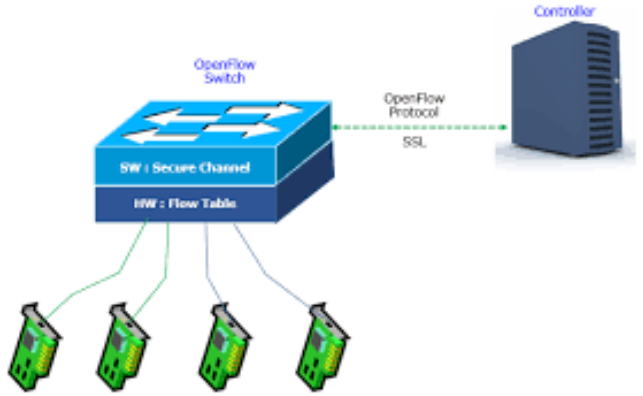
Alors:

- **Qu'est-ce exactement OpenFlow ?**
- **Qu'est-ce qu'un commutateur OpenFlow ?**
- **Comment un commutateur OpenFlow fonctionne-t-il pour améliorer l'agilité et l'évolutivité du réseau ?**



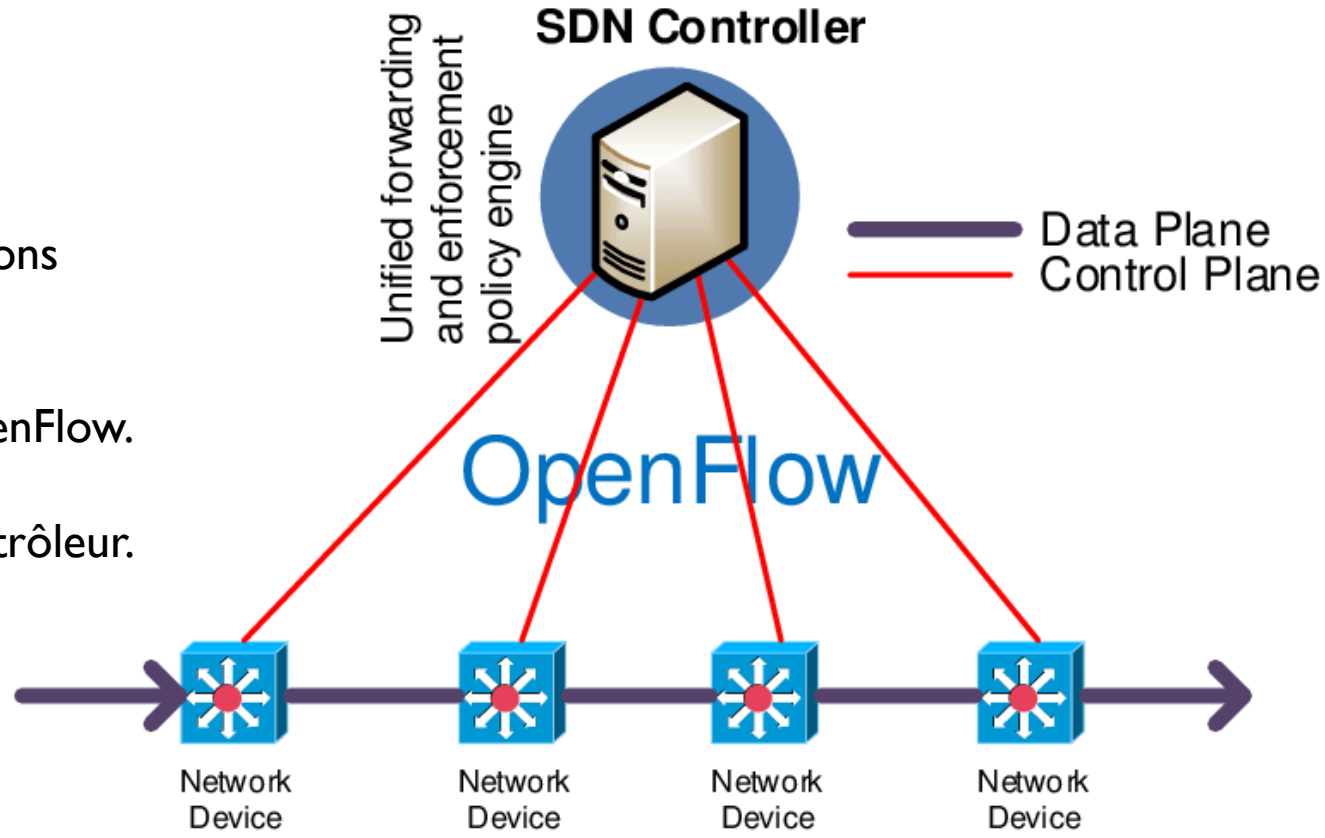
OpenFlow

OpenFlow est un protocole de communication central utilisé dans les réseaux définis par logiciel (SDN) pour permettre à un contrôleur SDN de communiquer avec les commutateurs et routeurs du réseau, leur dictant comment acheminer les paquets.



Architecture OpenFlow

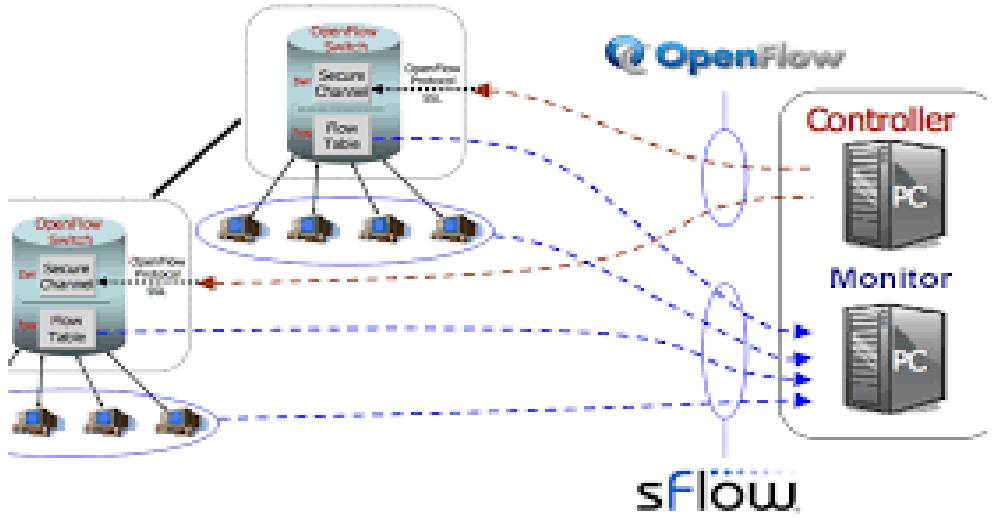
- **Switch OpenFlow**
  - Gère les **flow tables** et exécute des actions basées sur ces règles.
- **Contrôleur SDN**
  - Donne des instructions au switch via OpenFlow.
- **Canal OpenFlow**
  - Communication entre le switch et le contrôleur.



QU'EST-CE QU'OPENFLOW ET UN COMMUTATEUR OPENFLOW ?

- OpenFlow est un protocole réseau programmable pour un environnement SDN, utilisé pour la communication entre les commutateurs OpenFlow et les contrôleurs.

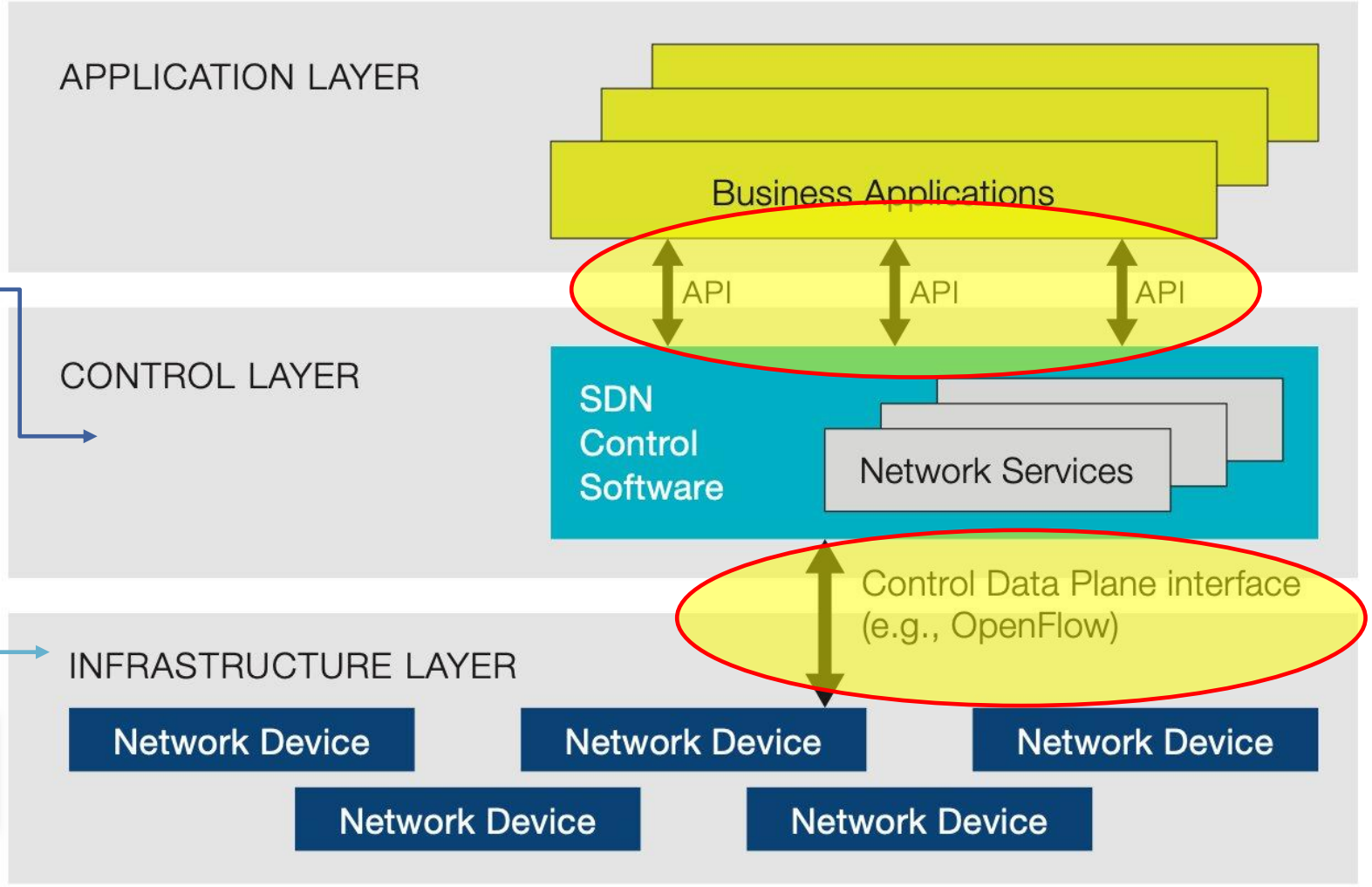
- OpenFlow sépare la programmation des dispositifs réseau du matériel sous-jacent et offre une méthode standardisée pour fournir un réseau centralisé et programmable qui peut s'adapter rapidement aux exigences changeantes du réseau.



SDN: Concepts fondamentaux d'OpenFlow

Plan de contrôle (Control Plane)
Exemple: Onos, Opendaylight

- Décide du chemin que les paquets doivent emprunter.
- Gère les politiques de routage, de sécurité, de QoS.



Plan de contrôle vs Plan de données
Mininet

Plan de données (Data Plane) :

- Exécute les décisions du plan de contrôle.
- Transfère les paquets selon les règles établies.

OVS, OpenFlow, ovs-vsctl, ovs-ofctl

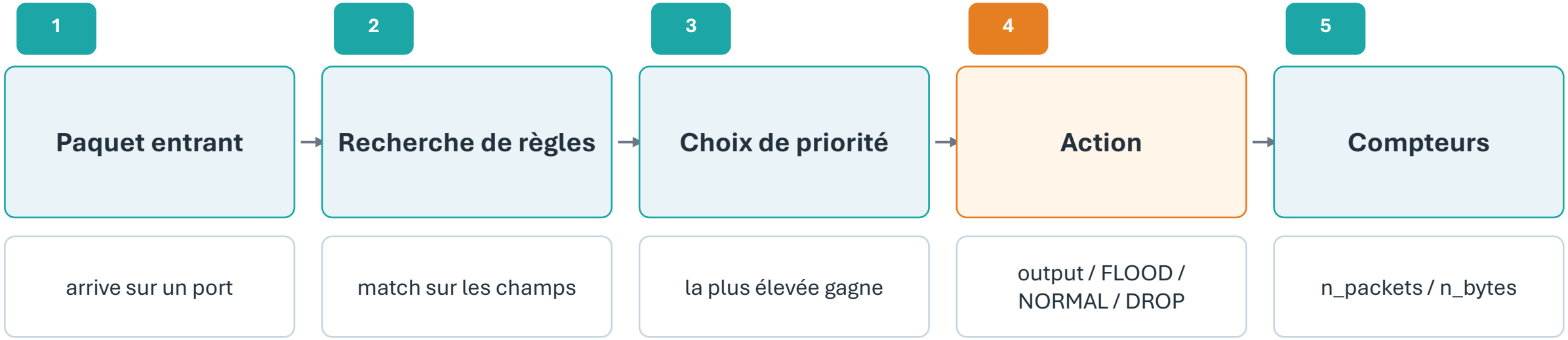
Quatre notions souvent confondues

Open vSwitch	OpenFlow	ovs-vsctl	ovs-ofctl
Le switch logiciel	**Le protocole**	**Configurer OVS**	**Programmer les flux**
ports, interfaces, datapath, tables	match → action dans le plan de données	bridge, ports, protocoles, fail-mode	add-flow, dump-flows, del-flows

Mémo : ovs-vsctl configure le switch ; ovs-ofctl configure le comportement OpenFlow du switch.

Comment une Flow Table traite un paquet ?

La logique fondamentale d'OpenFlow



Exemple — trois règles compatibles avec des rôles différents

Priorité	Match	Action
100	arp	FLOOD
200	ip,nw_dst=10.0.0.3	output:3
300	icmp,...,icmp_type=8	DROP

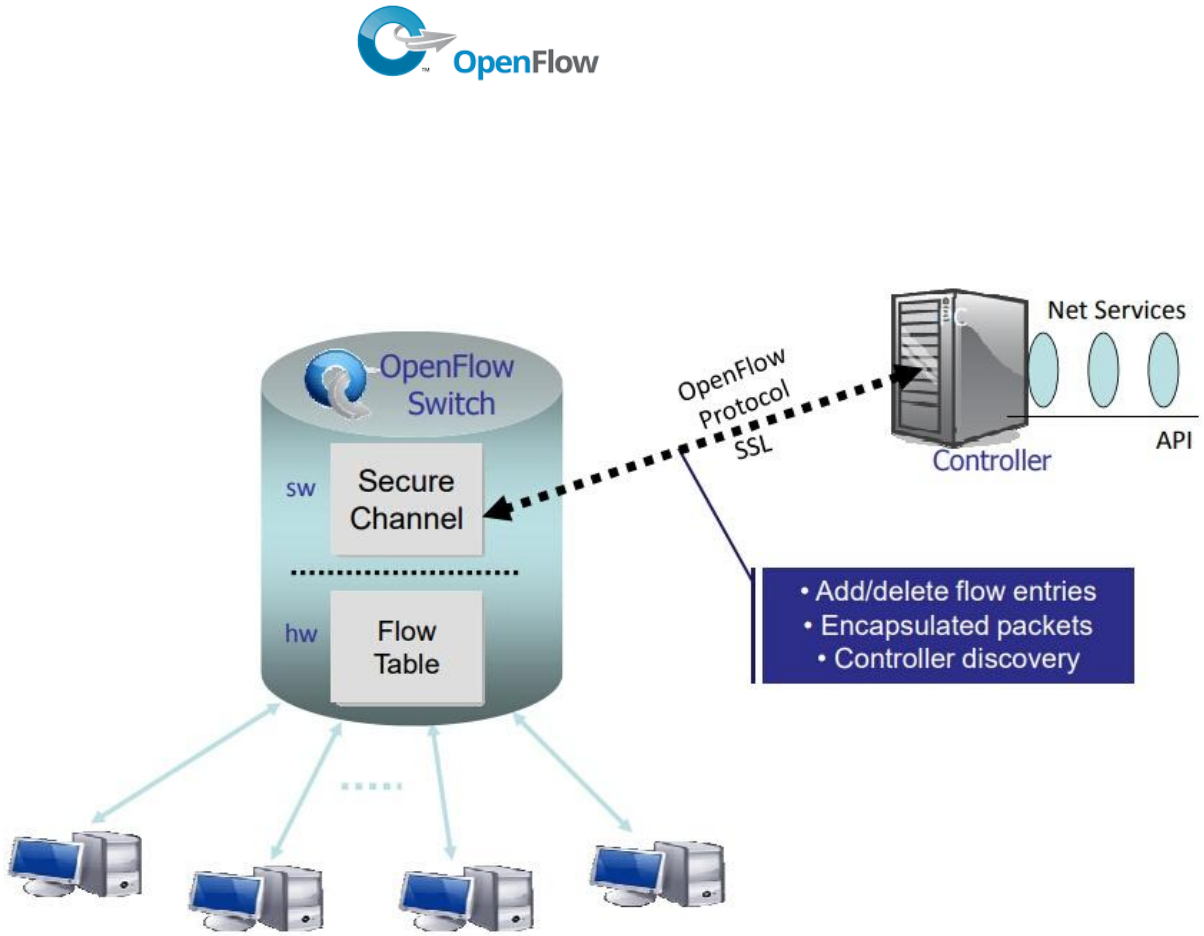
Sans règle applicable dans le scénario secure : aucune action de forwarding n'est exécutée.

OpenFlow Switch :

Le **switch OpenFlow** est l'équipement réseau qui reçoit et achemine les paquets de données. Il se compose de deux parties principales :

- **SW (software)** : C'est la partie logicielle qui gère la communication avec le contrôleur via un **canal sécurisé** (SSL).
- **HW (hardware)** : C'est la partie matérielle où sont exécutées les instructions via une **table de flux (flow table)**.

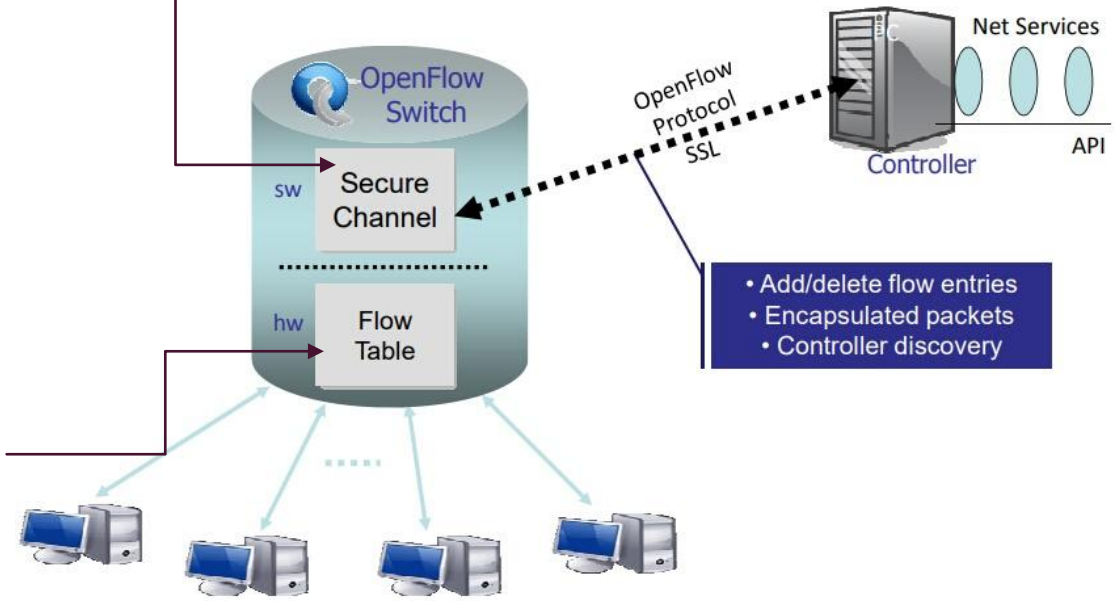
La table de flux contient des règles sur la manière de traiter les paquets en fonction des instructions reçues du contrôleur.



OpenFlow Switch

Canal sécurisé (Secure Channel) :
C'est le lien de communication crypté entre le switch et le contrôleur SDN. Il utilise des protocoles de sécurité comme **SSL** (Secure Sockets Layer) pour garantir la confidentialité et l'intégrité des échanges entre le switch et le contrôleur.

sh ovs-ofctl dump-flows s1 -O OpenFlow13



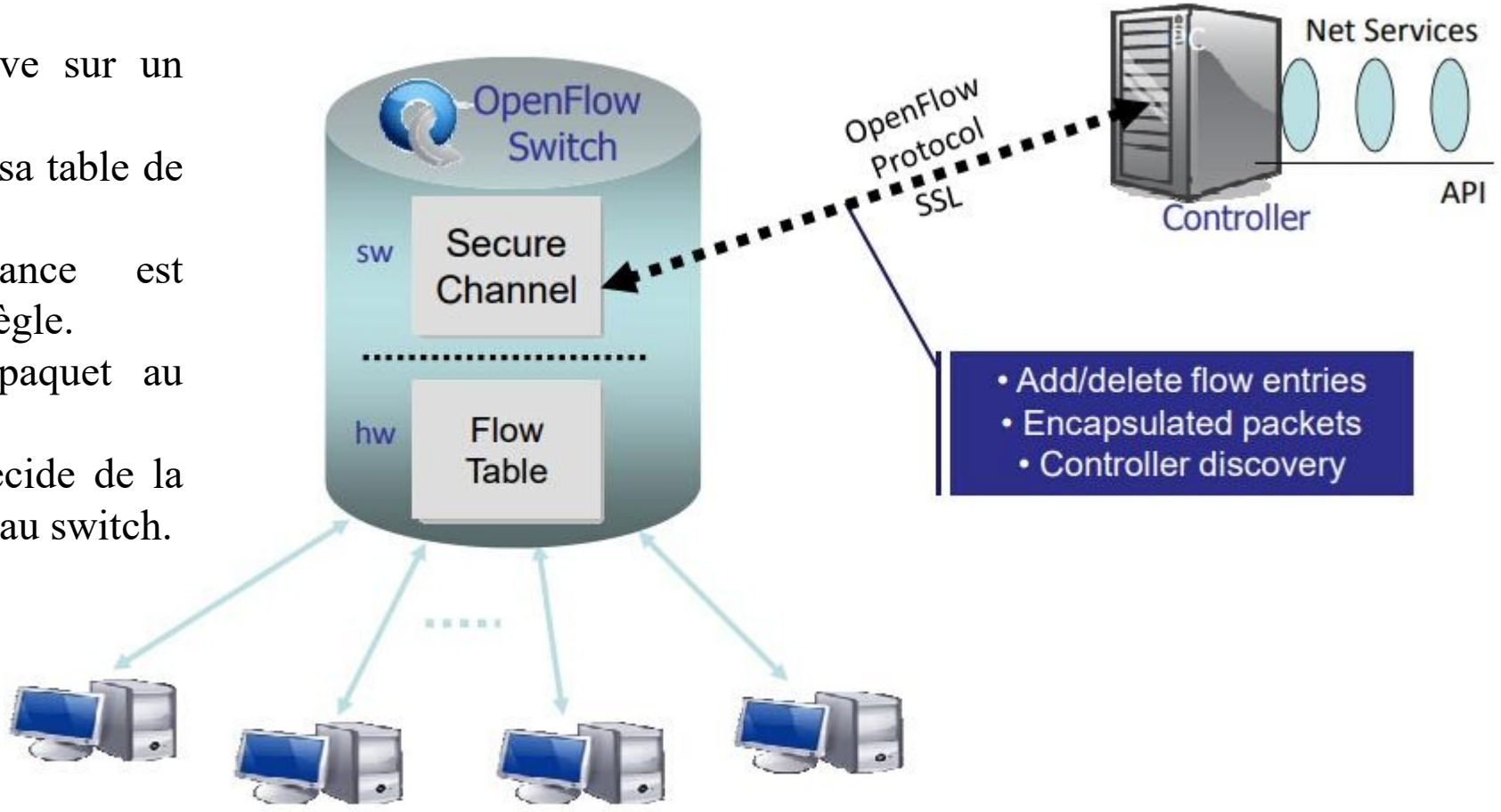
Flow Table (Table de flux) :
La table de flux est une base de données qui stocke les règles définies par le contrôleur. Ces règles dictent comment les paquets doivent être gérés (transmis, abandonnés, modifiés, etc.).
La table peut également inclure des actions à réaliser sur les paquets (par exemple, changer l'adresse de destination ou le port de sortie).

Le protocole OpenFlow et Switch Openflow: exemple OVS

Ce schéma présente le fonctionnement d'un **réseau SDN** (Software-Defined Networking) basé sur le protocole **OpenFlow**:

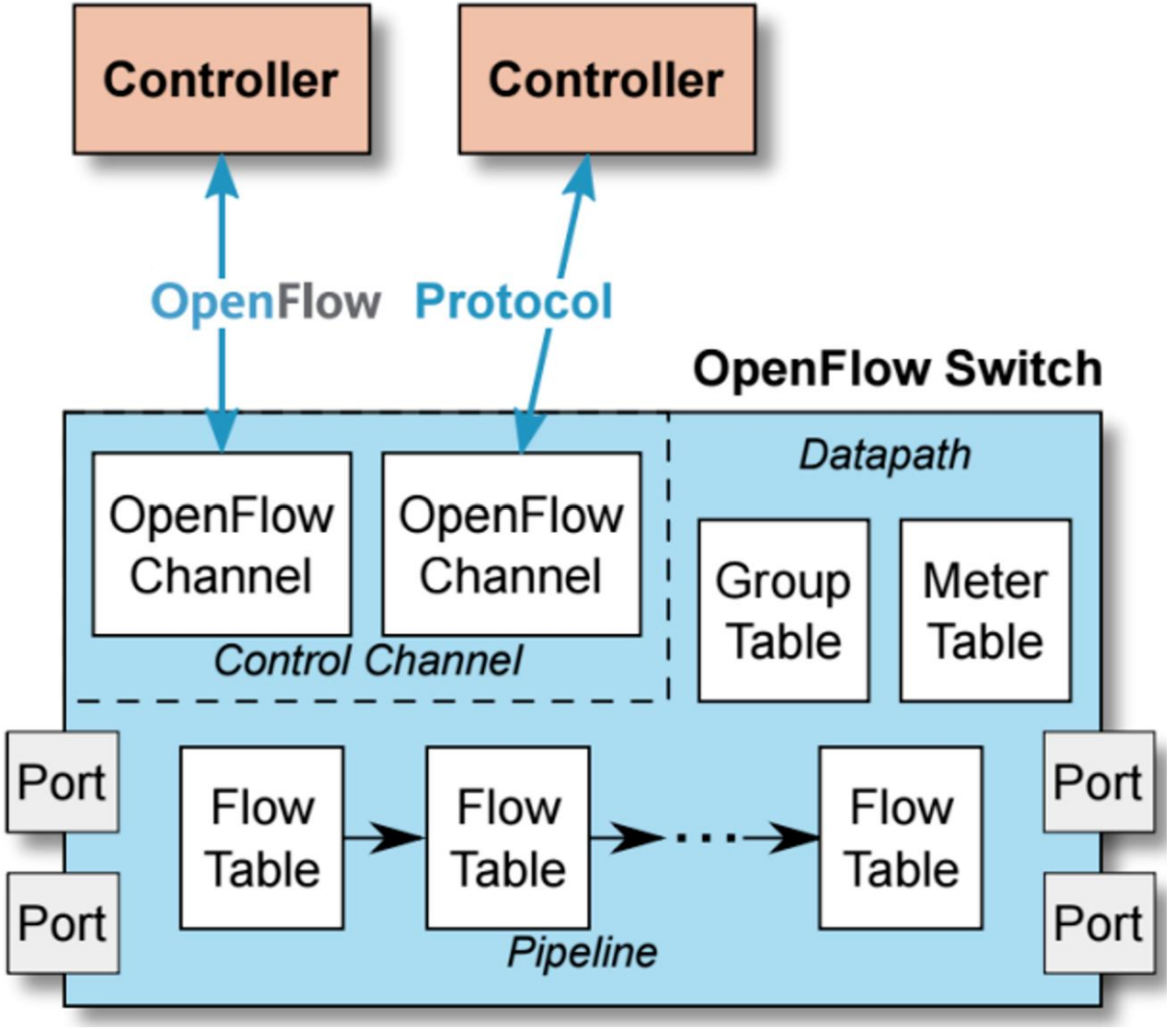
Fonctionnement d'OpenFlow : un exemple concret

- **Étape 1 :** Un paquet arrive sur un switch OpenFlow.
- **Étape 2 :** Le switch vérifie sa table de flux.
  - Si une correspondance est trouvée, il applique la règle.
  - Sinon, il envoie le paquet au contrôleur.
- **Étape 3 :** Le contrôleur décide de la règle à appliquer et l'envoie au switch.

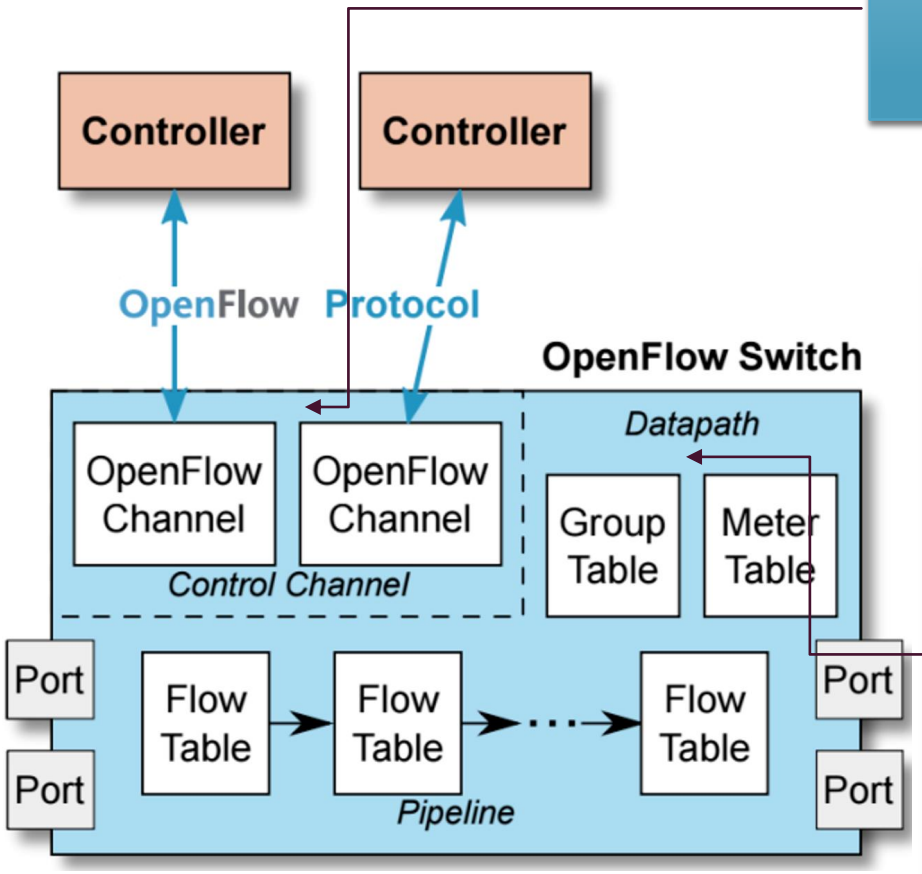


Commutateur OpenFlow (OpenFlow Switch) :

Ce schéma illustre l'architecture et le fonctionnement d'un **commutateur OpenFlow** dans un environnement SDN (Software-Defined Networking).



Commutateur OpenFlow (OpenFlow Switch) : Structure



Control Channel (Canal de contrôle) : Interface entre le commutateur et le contrôleur.

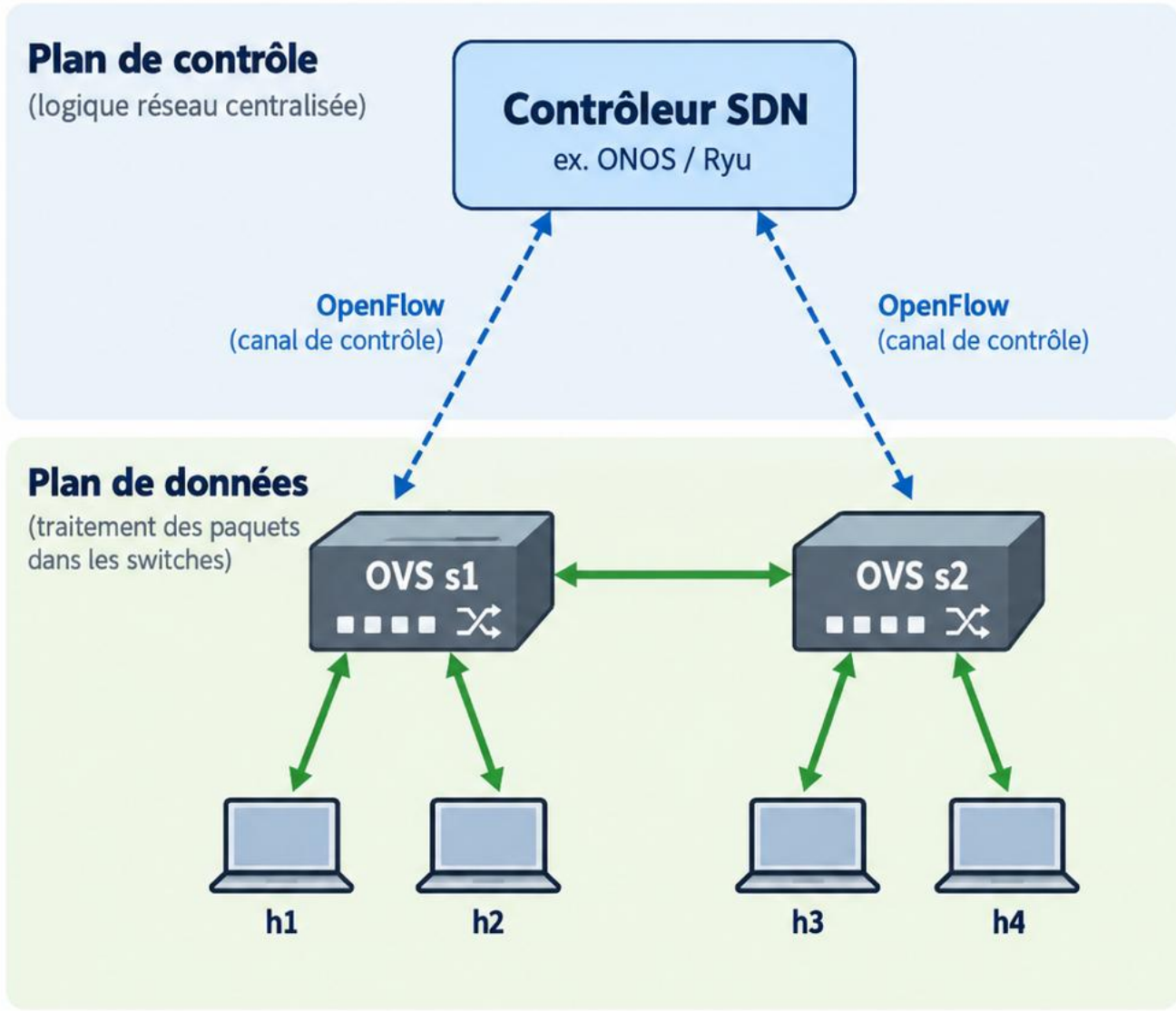
- Le **OpenFlow Channel** sert à échanger des messages de contrôle : par exemple, lorsque le commutateur consulte le contrôleur pour savoir comment gérer un paquet non reconnu.

Datapath (Chemin de données) :

Le plan où **les paquets de données sont réellement traités**:

- **Flow Table (Table de flux)** :
  - Chaque table de flux contient des règles (ou "flows") déterminent comment traiter un paquet.
  - Si un paquet correspond à une entrée de la table de flux, l'action correspondante est exécutée.
- **Group Table** :
  - Utilisée pour des actions avancées comme **l'équilibrage de charge** ou la **réplication des paquets**.
- **Meter Table** :
  - Gère la **limitation de débit** et la surveillance des performances du trafic.

OpenFlow avec contrôleur : architecture générale



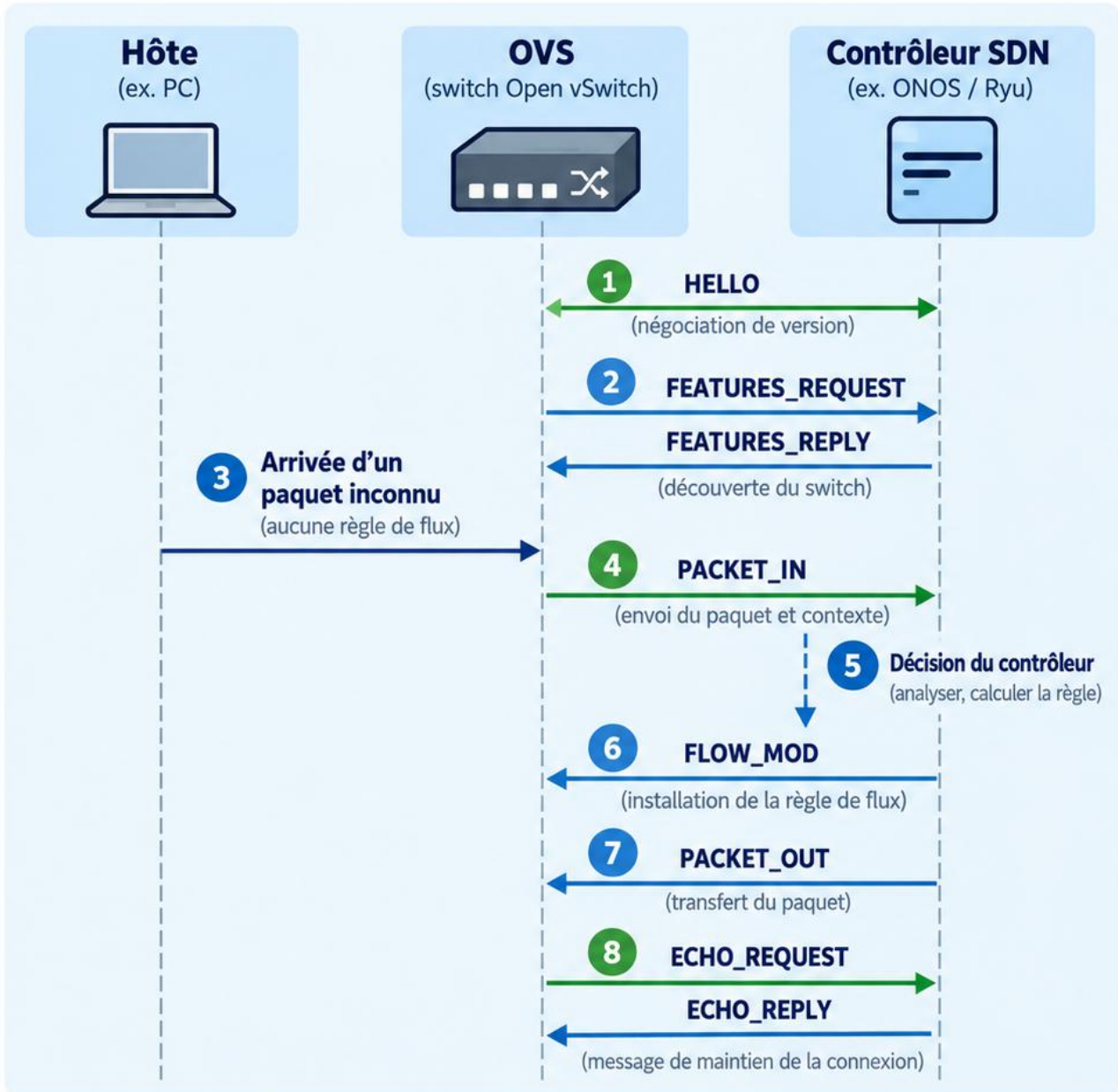
Principes

1. **Plan de contrôle centralisé**
   Le contrôleur a une vue globale du réseau.
2. **Plan de données dans les switches**
   Les switches (ex. OVS) traitent les paquets selon leurs tables de flux.
3. **Le contrôleur installe des règles de flux**
   Il configure les switches via le protocole OpenFlow.
4. **Les paquets sont traités selon les tables OpenFlow**
   Chaque switch applique les règles de flux pour transférer les paquets.

À retenir

1. Le contrôleur décide
2. OVS exécute
3. OpenFlow relie les deux

OpenFlow avec contrôleur : séquence des échanges



Lecture de la séquence

1. **Négociation de la version**
   OVS et le contrôleur s'échangent un message HELLO.
2. **Découverte du switch**
   Le contrôleur demande les capacités du switch (FEATURES_REQUEST) et reçoit la réponse (FEATURES_REPLY).
3. **Arrivée d'un paquet inconnu**
   Un hôte envoie un paquet pour lequel il n'existe pas de règle de flux dans OVS.
4. **Consultation du contrôleur**
   OVS envoie un PACKET_IN contenant le paquet et des informations de contexte.
5. **Décision du contrôleur**
   Le contrôleur analyse le paquet et choisit l'action à appliquer.
6. **Installation des règles**
   Le contrôleur envoie un FLOW_MOD pour installer la règle de flux dans OVS.
7. **Transmission du paquet**
   Le contrôleur envoie un PACKET_OUT pour que OVS transfère le paquet.
8. **Maintien de la connexion**
   Des messages ECHO_REQUEST / ECHO_REPLY sont échangés régulièrement.

À retenir

1. Le contrôleur ne traite pas tous les paquets
2. FLOW_MOD installe des règles
3. PACKET_IN apparaît selon la politique configurée

Échanges OpenFlow entre ONOS et OVS (OpenFlow 1.3)



Message OpenFlow	Sens	Rôle
HELLO	ONOS ↔ OVS	Négocier la version OpenFlow
FEATURES_REQUEST	ONOS → OVS	Demander les caractéristiques du switch
FEATURES_REPLY	OVS → ONOS	Retourner l'identifiant DPID et les capacités
MULTIPART_REQUEST	ONOS → OVS	Demander les ports, flux ou statistiques
MULTIPART_REPLY	OVS → ONOS	Renvoyer les informations demandées
PACKET_IN	OVS → ONOS	Envoyer un paquet au contrôleur selon les règles configurées
FLOW_MOD	ONOS → OVS	Ajouter, modifier ou supprimer des règles de flux
PACKET_OUT	ONOS → OVS	Demander au switch d'émettre un paquet
ECHO_REQUEST / ECHO_REPLY	ONOS ↔ OVS	Vérifier que la connexion de contrôle est active
BARRIER_REQUEST / BARRIER_REPLY	ONOS ↔ OVS	Synchroniser le traitement des messages OpenFlow

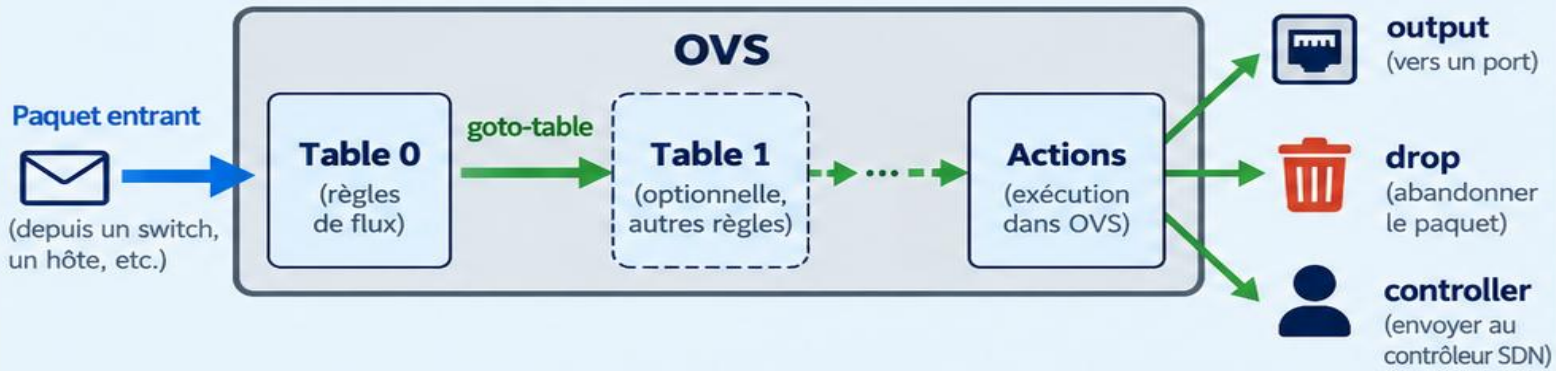
À retenir

- Un **PACKET_IN** n'est pas automatique : il dépend des règles installées.
- **FLOW_MOD** programme la table de flux de l'OVS.
- **PACKET_OUT** sert à faire sortir immédiatement un paquet depuis le switch.

Tables OpenFlow dans OVS : principe de fonctionnement

Pipeline de traitement d'un paquet dans OVS

(tables OpenFlow enchaînées)



Contenu d'une table OpenFlow

(exemple d'entrées de flux dans une table d'OVS)

Une entrée de flux (flow entry)

Match (champs du paquet)	Priority (priorité)	Counters (n_packets, n_bytes, ...)	Instructions / Actions (ex. output, drop, controller, goto-table)

#	Priorité	Match (exemples)	Compteurs (exemples)	Instructions / Actions
1	200	ARP	n_packets=1250 n_bytes=75000	FLOOD
2	150	IP, nw_dst=10.0.0.2	n_packets=340 n_bytes=20480	output:2
3	120	ICMP	n_packets=98 n_bytes=7840	CONTROLLER
4	0	* (tous les paquets)	n_packets=5600 n_bytes=3.2M	DROP

Principes

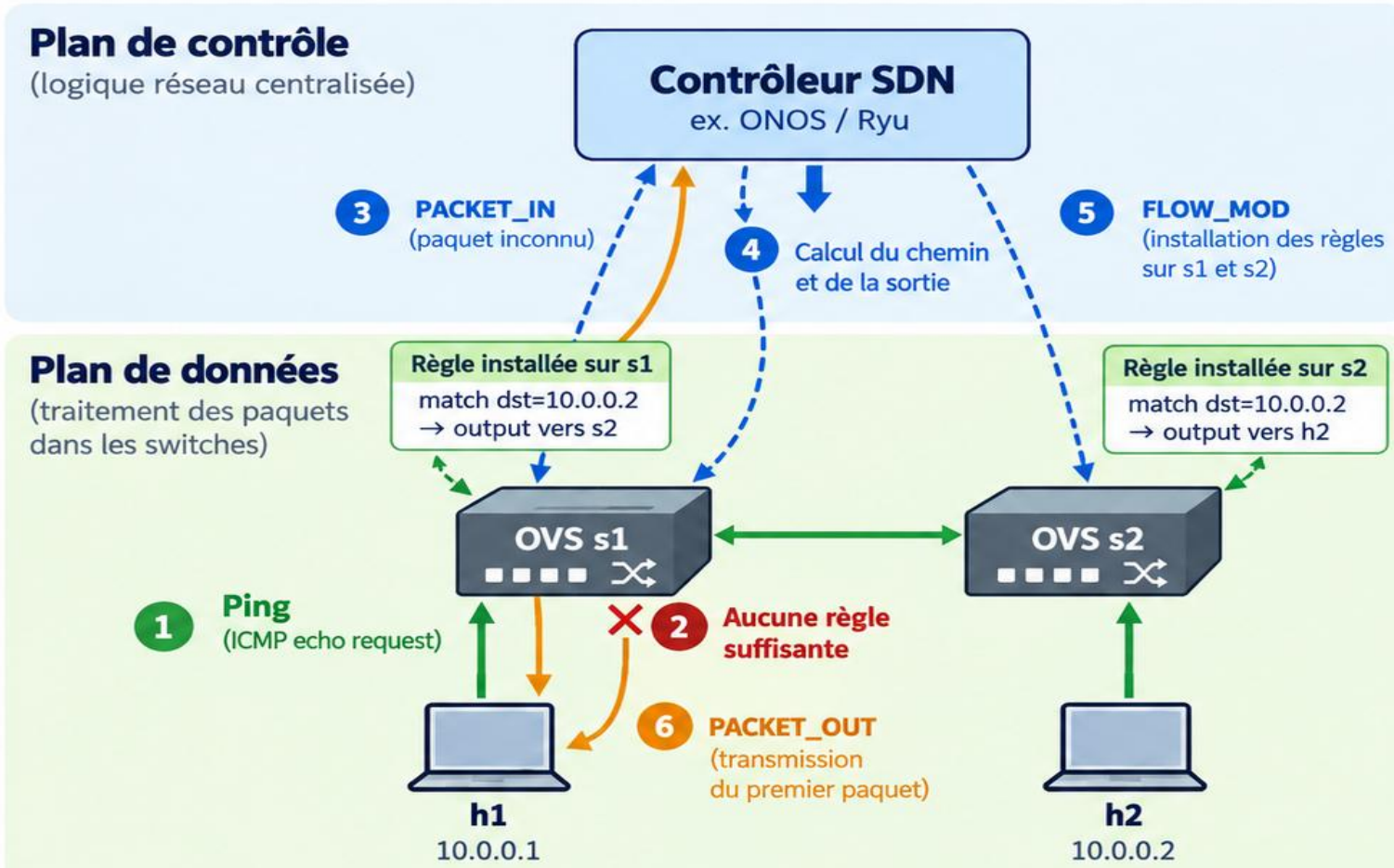
1. **Le switch compare le paquet aux règles**
   Les champs du paquet sont comparés avec le champ Match de chaque entrée de la table.
2. **La priorité la plus élevée l'emporte**
   Parmi les entrées qui correspondent, celle avec la priorité la plus élevée est sélectionnée.
3. **Si aucune règle pertinente : action de secours**
   Si aucune entrée ne correspond, une action par défaut (ex. DROP) est utilisée.
4. **Les compteurs suivent n_packets et n_bytes**
   Chaque entrée maintient des compteurs de paquets et d'octets, utiles pour le monitoring et le diagnostic.

À retenir

1. Match + priorité = décision
2. Les actions sont exécutées dans OVS
3. Les compteurs aident au diagnostic

Scénario 1 : paquet inconnu puis installation des flux

Plan de contrôle
(logique réseau centralisée)



Pourquoi c'est utile ?

1. **Programmation dynamique**
   Le contrôleur installe les règles à la demande, selon les paquets observés.
2. **Logique centralisée**
   Le contrôleur prend les décisions de chemin à partir d'une vue globale du réseau.
3. **Délai sur le premier paquet**
   Le premier paquet déclenche un aller-retour avec le contrôleur, les suivants sont traités directement dans les switches.

À retenir

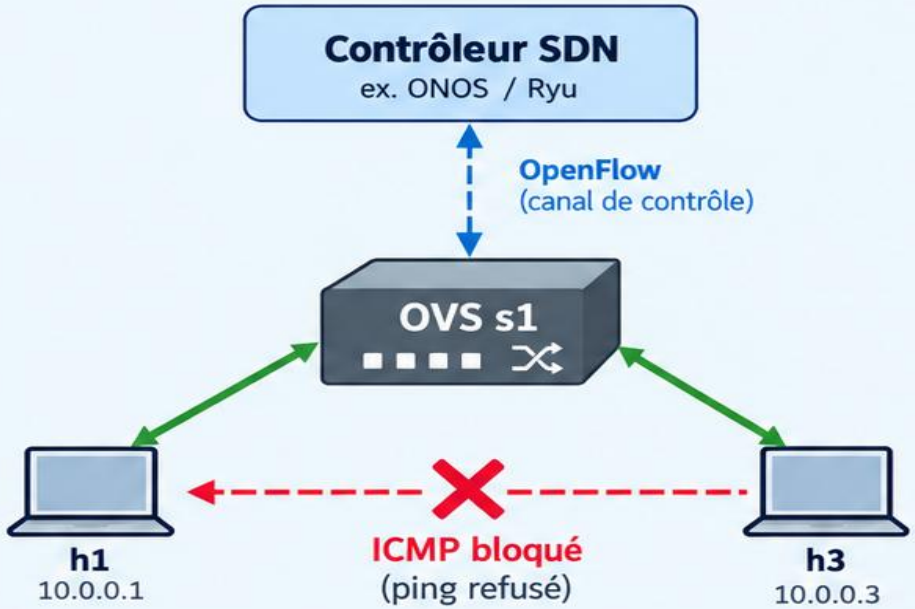
1. Le premier paquet peut déclencher le contrôleur
2. Les suivants passent directement dans OVS
3. Le chemin est ensuite déjà programmé

Étapes du scénario

1. **h1 envoie un ping vers h2**
   Un paquet ICMP est envoyé de 10.0.0.1 vers 10.0.0.2.
2. **s1 ne trouve pas de règle suffisante**
   Aucune entrée de table ne correspond au paquet.
3. **s1 envoie PACKET_IN au contrôleur**
   Le switch transmet le paquet et ses informations au contrôleur.
4. **Le contrôleur calcule le chemin et la sortie**
   Il décide d'envoyer le trafic de h1 vers h2 via s1 → s2.
5. **FLOW_MOD installe les règles sur s1 et s2**
   Le contrôleur envoie des règles de flux aux deux switches.
6. **PACKET_OUT / output transmet le premier paquet**
   Le contrôleur demande au switch d'envoyer le paquet original vers s2.

Scénarios 2 et 3 : sécurité et pilotage du trafic

Scénario 2 – Bloquer l'ICMP entre h3 et h1



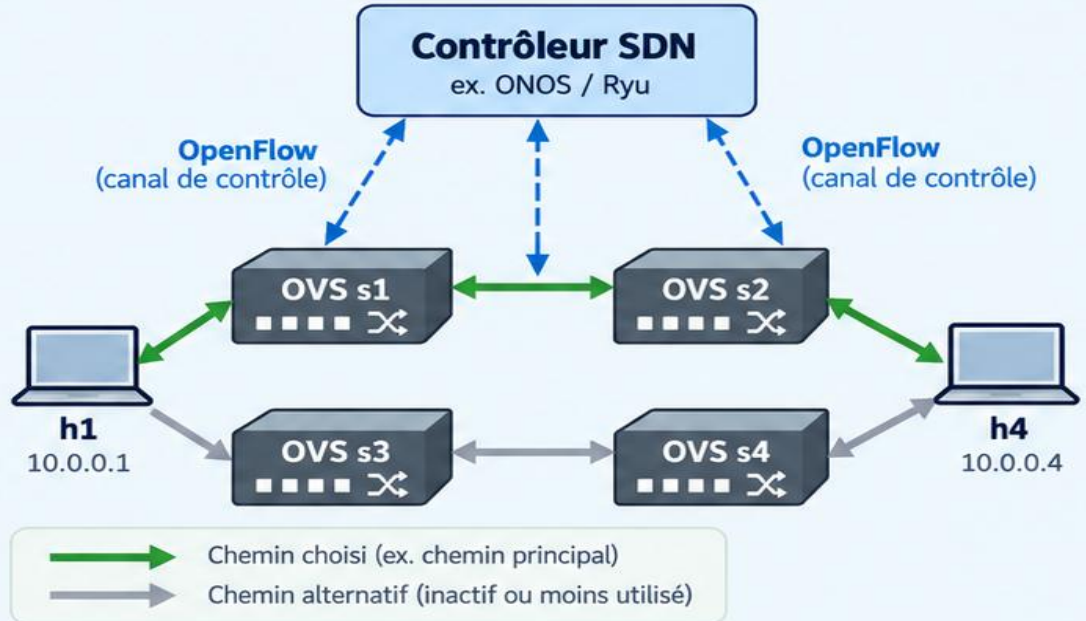
Règle de flux installée

```
priority=300, icmp,
nw_src=10.0.0.3,
nw_dst=10.0.0.1,
actions=DROP
```

Principes

1. **Politique de sécurité**
   On peut bloquer certains protocoles ou hôtes.
2. **Flux à haute priorité**
   Une règle spécifique avec une priorité élevée s'applique en premier.
3. **Le ping est refusé**
   Le trafic ICMP de h3 vers h1 est bloqué par le switch.

Scénario 3 – Choisir un chemin ou équilibrer le trafic



Exemple de règle de flux

```
match dst=10.0.0.4
-> output:port 3
```

Principes

1. **Choix du chemin**
   Le contrôleur installe la règle pour envoyer le trafic vers le bon port (chemin choisi).
2. **Adaptation dynamique**
   Le contrôleur peut changer le chemin selon la charge ou la panne (ex. basculer vers s3–s4).

À retenir

1. OpenFlow sert aussi à filtrer.
2. La priorité permet de surcharger une règle.
3. Le contrôleur choisit les sorties.
4. Les scénarios dépendent de la politique réseau.

Anatomie d’une règle OpenFlow

Lire la syntaxe sans la mémoriser par cœur

```
ovs-ofctl -O OpenFlow13 add-flow s1 "priority=300,icmp,nw_src=10.0.0.1,nw_dst=10.0.0.3,icmp_type=8,actions=drop"
```

-O OpenFlow13	Version du protocole	**add-flow s1**	Ajouter une entrée sur s1
priority=300	Priorité de la règle	**icmp**	Paquet ICMP
nw_src / nw_dst	Source et destination IP	**icmp_type=8**	Echo Request
actions=drop	Supprimer le paquet		

Principe : MATCH + PRIORITÉ + ACTION. Les compteurs permettent ensuite de vérifier combien de paquets ont utilisé la règle.

TP OpenFlow 1 : principe général du TP

Terminal 1 : Mininet

Génère du trafic
- ping (ex. h1 → h3)
- curl (ex. h1 → h2)
- autres tests ...

Terminal 2 : Ubuntu
+ sudo ovs-ofctl / ovs-vsctl

Programme la table de flux de s1 (règles OpenFlow)
- ovs-ofctl add-flow ...
- ovs-vsctl ...

Wireshark graphique

Observe les paquets sur certaines interfaces du switch
- s1-eth1 (trafic de h1)
- s1-eth3 (trafic de h3)

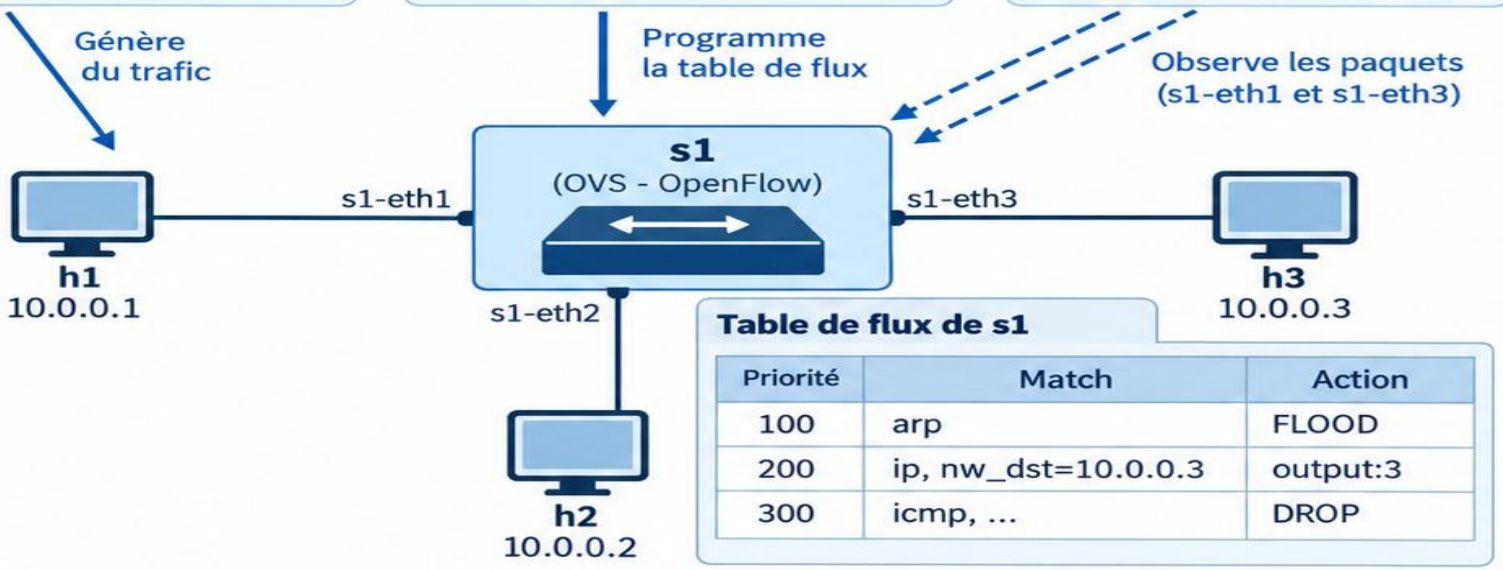


Table de flux de s1

Priorité	Match	Action
100	arp	FLOOD
200	ip, nw_dst=10.0.0.3	output:3
300	icmp, ...	DROP

Objectifs

1. Comprendre le rôle de la table de flux
2. Programmer OVS manuellement
3. Observer l'entrée et la sortie des paquets
4. Préparer la transition vers un contrôleur SDN

Idée clé

- Le switch n'invente pas la politique
- Les règles déterminent le trafic autorisé
- Wireshark prouve le comportement du plan de données

Flux commentés : priorités, ACL ICMP et ACL TCP

Scénario 1 — Bloquer le ping h1 -> h3



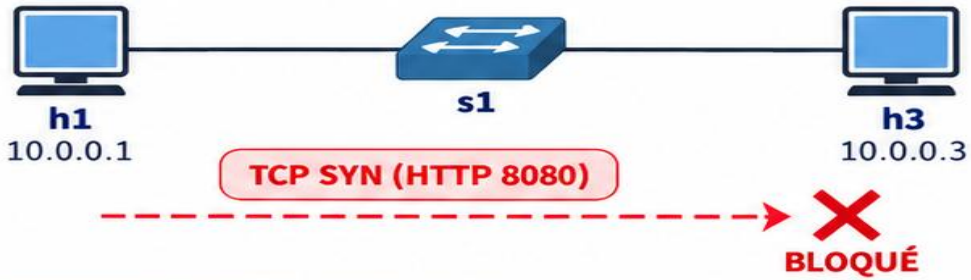
Règle dans la Flow Table (ACL) :

```
priority=300, icmp, nw_src=10.0.0.1,
nw_dst=10.0.0.3, actions=drop
```

Cette règle l'emporte sur les flux IPv4 de priorité 200.

Avec **icmp_type=8**, seul l'Echo Request est bloqué ; le ping initié par h3 peut encore réussir.

Scénario 2 — Bloquer le service HTTP 8080



Règle dans la Flow Table (ACL) :

```
priority=350, tcp, nw_src=10.0.0.1,
nw_dst=10.0.0.3, tcp_dst=8080, actions=drop
```

Avant l'ACL : HTTP 200 OK
Après l'ACL : SYN visible à l'entrée, absent à la sortie.

Filtres utiles dans Wireshark

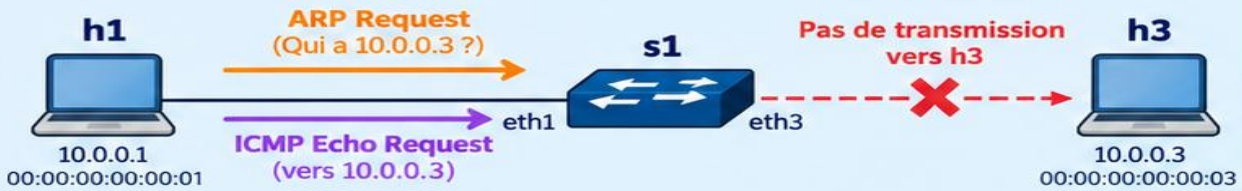
```
icmp
tcp.port == 8080
ip.addr == 10.0.0.1 || ip.addr == 10.0.0.3
```

Leçons du TP

1. La règle la plus spécifique et prioritaire gagne.
2. **DROP** coupe le trafic ciblé.
3. **Wireshark** confirme l'effet de la Flow Table.
4. Un futur contrôleur SDN automatisera ces opérations.

Flux commentés : table vide, ARP seul, forwarding IPv4

1. Table vide



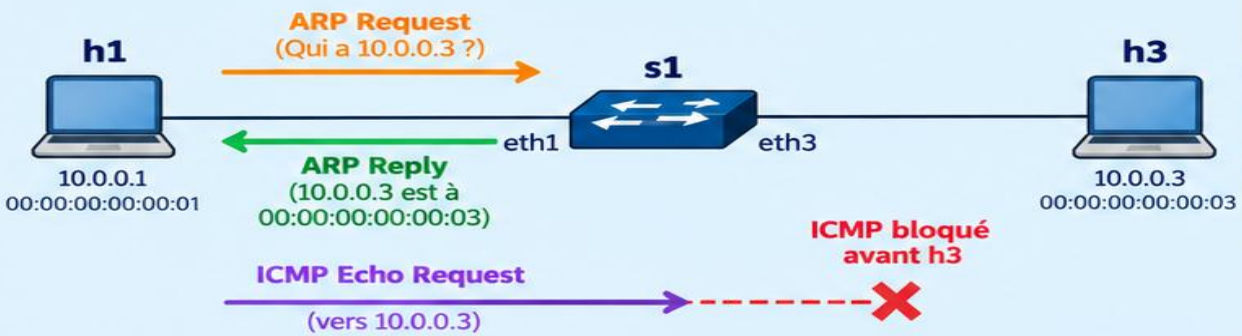
Wireshark sur s1

s1-eth1 : **trame visible**
s1-eth3 : **aucune retransmission**

Explication

Sans règle utile, le switch reçoit mais ne transmet pas.

2. ARP uniquement



Règle installée sur s1

```
priority=100, arp,
actions=FLOOD
```

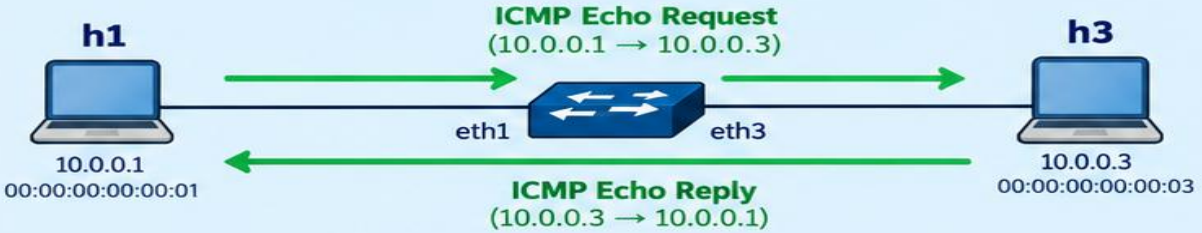
Wireshark sur s1

s1-eth1 : **trames ARP visibles**
s1-eth3 : **trames ARP visibles**
trames ICMP : **aucune retransmission**

Explication

ARP fonctionne, mais pas encore IPv4.

3. Forwarding IPv4



Règles installées sur s1

```
ip,nw_dst=10.0.0.1, actions=output:1
ip,nw_dst=10.0.0.2, actions=output:2
ip,nw_dst=10.0.0.3, actions=output:3
```

Wireshark sur s1

Le paquet est **visible à l'entrée et à la sortie.**

À retenir

1. **ARP résout les MAC**
   ARP permet d'associer une adresse IP à une adresse MAC (diffusion ARP).

2. **Les règles IPv4 décident du port de sortie**
   Le switch utilise les champs IPv4 (ex. nw_dst) pour choisir le port de sortie.

3. **La priorité départage les règles**
   En cas de correspondance multiple, c'est la règle avec la plus haute priorité qui est appliquée.

4. **Une même topologie change de comportement selon la Flow Table**
   Avec les mêmes hôtes et le même switch, le comportement dépend des règles installées dans la table de flux.

TP OpenFlow 1 : syntaxe des commandes — qui tape quoi ?

Terminal 1 : Mininet

Lance la topologie et génère le trafic (côté hôtes).

1 Lancer Mininet

```
sudo mn --topo single,3 \
  --switch ovsk,protocols=OpenFlow13 \
  --controller none
```

Détail des options :

- `--topo single,3` = 1 switch + 3 hôtes (h1, h2, h3)
- `--switch ovsk` = Open vSwitch (OVS)
- `protocols=OpenFlow13` = force OpenFlow 1.3
- `--controller none` = pas de contrôleur

2 Commandes dans le CLI Mininet

```
h1 ping -c 2 h3
```

- Envoie 2 paquets ICMP de h1 vers h3.

```
pingall
```

- Envoie des pings entre tous les hôtes.

```
h3 python3 -m http.server 8080
```

- Lance un serveur HTTP sur h3 (port 8080).

```
h1 curl -I http://10.0.0.3:8080
```

- Depuis h1, envoie une requête HTTP (en-têtes uniquement) vers le serveur de h3.

Terminal 2 : Ubuntu + sudo

Configure et interroge le switch Open vSwitch.

1 Mettre le switch en mode sécurisé

```
sudo ovs-vsctl set-fail-mode s1 secure
```

- `sudo` : exécute la commande en tant que root.
- `ovs-vsctl` : outil de configuration d'Open vSwitch.
- `set-fail-mode s1 secure` : si le contrôleur est absent, le switch bloque les paquets (mode sécurisé).

2 Forcer OpenFlow 1.3 sur le switch

```
sudo ovs-vsctl set bridge s1 \
     protocols=OpenFlow13
```

- `set bridge s1` : configure le pont (switch) s1.
- `protocols=OpenFlow13` : n'autorise que le protocole OpenFlow 1.3.

3 Afficher les informations du switch

```
sudo ovs-ofctl -O OpenFlow13 show s1
```

- `ovs-ofctl` : outil OpenFlow pour interroger le switch.
- `-O OpenFlow13` : utilise la syntaxe OpenFlow 1.3.
- `show s1` : affiche les ports et les informations du switch.

4 Afficher la table de flux

```
sudo ovs-ofctl -O OpenFlow13 dump-flows s1
```

- `dump-flows s1` : affiche les règles de la Flow Table installées sur le switch.
- Au début du TP, la table doit être vide (aucune règle).

Fenêtre 3 : Wireshark graphique

Analyse les paquets sur les ports du switch.

1 Lancer Wireshark

```
wireshark &
```

- Ouvre Wireshark en tâche de fond.

2 Sélectionner les interfaces



- Menu **Capture ▸ Options**.
- Cocher les interfaces `s1-eth1` et `s1-eth3` du switch (s1).
- Laisser les autres par défaut.

3 Filtre d'affichage

```
arp || icmp || tcp.port == 8080
```

- Affiche uniquement les paquets ARP, ICMP et le trafic TCP sur le port 8080.
- Dans la liste des paquets, observer l'entrée (s1-eth1) et la sortie (s1-eth3).

À retenir

1. **Mininet génère le trafic**
   Crée la topologie et les hôtes, et permet de tester (ping, HTTP...).
2. **ovs-vsctl configure le switch**
   Permet de définir le mode sécurisé et la version d'OpenFlow.
3. **ovs-ofctl programme la Flow Table**
   Permet d'afficher et d'ajouter des règles OpenFlow.
4. **Wireshark vérifie le résultat**
   Capture les paquets sur les ports du switch et confirme le comportement.

TP OpenFlow 1 : syntaxe de ovs-ofctl expliquée

1 Ajouter une règle de forwarding IPv4

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=200,ip,nw_dst=10.0.0.3,actions=output:3"
```

- `sudo` : Privilèges administrateur (nécessaire pour gérer Open vSwitch)
- `ovs-ofctl` : Outil en ligne de commande pour gérer OpenFlow sur Open vSwitch
- `-O OpenFlow13` : Version du protocole OpenFlow (utilisation de OpenFlow 1.3)
- `add-flow` : Ajouter une règle dans la table de flux
- `s1` : Bridge ciblé (ici s1)
- `"priority=200,ip,nw_dst=10.0.0.3,actions=output:3"` :
  - `priority=200` : Priorité de la règle (200)
  - `ip` : Trafic IPv4 (champs L3)
  - `nw_dst=10.0.0.3` : Adresse IP de destination (10.0.0.3)
  - `actions=output:3` : Action : envoyer le paquet vers le port 3 du switch

2 Ajouter une règle pour ARP (diffusion)

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=100, arp, actions=FLOOD"
```

- `sudo` : Privilèges administrateur
- `ovs-ofctl` : Outil OpenFlow
- `-O OpenFlow13` : Version du protocole (1.3)
- `add-flow` : Ajouter une règle
- `s1` : Bridge ciblé
- `"priority=100, arp, actions=FLOOD"` :
  - `priority=100` : Priorité (100)
  - `arp` : Trafic ARP (trames ARP)
  - `actions=FLOOD` : Action : diffusion sur les ports appropriés (FLOOD)

3 Consulter et supprimer des règles

```
sudo ovs-ofctl -O OpenFlow13 dump-flows s1
```

→ **`dump-flows` :** lire et afficher les règles installées sur le bridge s1.

```
sudo ovs-ofctl -O OpenFlow13 del-flows s1
```

→ **`del-flows` :** supprimer toutes les règles du bridge s1.

```
sudo ovs-ofctl -O OpenFlow13 --strict del-flows s1 "priority=300,icmp,nw_src=10.0.0.1,nw_dst=10.0.0.3"
```

- `--strict` : supprimer exactement la règle qui correspond au match et à la priorité.
- `del-flows` : supprimer une règle (avec --strict)
- `s1` : Bridge ciblé
- `"priority=300,icmp,nw_src=10.0.0.1,nw_dst=10.0.0.3"` : Critères de correspondance : priorité, protocole ICMP, adresse IP source et destination.

Règles de syntaxe

1 Champs L3/L4 et protocole

- Les champs de niveau 3 (ex. `nw_src`, `nw_dst`) nécessitent de spécifier le protocole correspondant (ex. `ip`).
- Pour ICMP : utiliser `icmp`.
- Pour TCP ou UDP : utiliser `tcp` ou `udp` (puis les champs `tp_src`, `tp_dst` si besoin).

2 Plusieurs règles peuvent correspondre

- Un paquet peut correspondre à plusieurs règles.
- La règle avec **la priorité la plus haute est appliquée.**

3 Actions fréquentes

- `output:N` : envoyer le paquet vers le port N.
- `FLOOD` : diffusion sur les ports appropriés.
- `NORMAL` : traitement normal par le switch (comme un switch classique).
- `drop` : jeter le paquet (ne rien faire).

Exemples de règles courantes

Match (critères)	Action	Effet / Description
`arp`	`FLOOD`	Diffusion des trames ARP sur les ports appropriés pour permettre la résolution d'adresse MAC.
`ip,nw_dst=10.0.0.3`	`output:3`	Envoi des paquets IPv4 destinés à 10.0.0.3 vers le port 3 (ex. vers l'hôte h3).
`icmp,nw_src=10.0.0.1,nw_dst=10.0.0.3`	`drop`	Blocage des paquets ICMP provenant de 10.0.0.1 et destinés à 10.0.0.3.

TP OpenFlow 1 : exemples de commandes commentées

1 Mettre le switch en mode secure et OF1.3

```
sudo ovs-vsctl set-fail-mode s1 secure
```

→ **secure** = sans politique de secours

```
sudo ovs-vsctl set bridge s1 protocols=OpenFlow13
```

→ **protocols=OpenFlow13** = force la version utilisée

2 Autoriser ARP uniquement

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 \
"priority=100,arp,actions=FLOOD"
```

→ permet la résolution ARP mais pas encore IPv4.

3 Forwarding IPv4 par destination

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 \
"priority=200,ip,nw_dst=10.0.0.1,actions=output:1"
```

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 \
"priority=200,ip,nw_dst=10.0.0.2,actions=output:2"
```

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 \
"priority=200,ip,nw_dst=10.0.0.3,actions=output:3"
```

→ chaque destination IP est envoyée vers le bon port.

4 ACL de blocage

Blocage ICMP (Echo Request)

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 \
"priority=300,icmp,nw_src=10.0.0.1,nw_dst=10.0.0.3,
icmp_type=8,actions=drop"
```

→ bloque seulement les Echo Request h1 → h3.

Blocage TCP (port 8080)

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 \
"priority=350,tcp,nw_src=10.0.0.1,nw_dst=10.0.0.3,
tcp_dst=8080,actions=drop"
```

→ bloque le service HTTP sur le port 8080.

Méthode à suivre dans le TP

1. **Installer / supprimer les flux**
   Utiliser ovs-ofctl pour ajouter ou supprimer des règles.
2. **Générer le trafic avec ping ou curl**
   Tester la connectivité et les services.
3. **Observer dans Wireshark**
   Vérifier les trames (ARP, ICMP, TCP, ...) et leur passage.
4. **Vérifier avec dump-flows et les compteurs**
   Utiliser ovs-ofctl dump-flows s1 et vérifier les compteurs de paquets.

Commutateur OpenFlow (OpenFlow Switch) : Étape par étape du fonctionnement du commutateur OpenFlow :

1. Arrivée d'un paquet au commutateur :

- Un paquet entre par l'un des **ports** du commutateur (indiqué à gauche dans le schéma).
- Le commutateur doit maintenant décider **comment traiter ce paquet** (par exemple : le transmettre, le bloquer, ou le modifier).

2. Vérification dans les Flow Tables :

- Le paquet est comparé aux règles présentes dans les **tables de flux (Flow Tables)** du pipeline.
- Chaque table de flux contient des **entrées** avec des conditions (ex. : adresse IP, type de protocole) et des **actions** (ex. : envoyer vers un port spécifique, modifier l'en-tête).

1. **Cas 1 : Le paquet correspond à une règle**
   - Si une règle dans la Flow Table correspond au paquet, l'action associée est exécutée (ex. : envoyer le paquet à un port particulier).
2. **Cas 2 : Le paquet ne correspond à aucune règle**
   - Si le paquet ne correspond à aucune règle, il est envoyé au **contrôleur SDN** via le **protocole OpenFlow**.

3. Consultation du contrôleur (via OpenFlow) :

- Le contrôleur SDN (comme ONOS ou OpenDaylight) reçoit le paquet et décide **quoi faire**.
- Le contrôleur peut :
  - Envoyer une **réponse immédiate** sur le traitement du paquet (ex. : l'envoyer vers un port spécifique).
  - **Ajouter une nouvelle règle** dans la Flow Table du commutateur, afin de gérer les prochains paquets similaires **sans demander au contrôleur**.

Commutateur OpenFlow (OpenFlow Switch) : Structure

4. Mise à jour dynamique des tables de flux :

- Si le contrôleur ajoute une nouvelle règle dans le commutateur, celle-ci est stockée dans une **Flow Table**.
- À l'avenir, lorsque des paquets similaires arrivent, le commutateur peut **les traiter directement sans contacter le contrôleur**, ce qui réduit le temps de traitement.

5. Rôles des autres tables (Group Table, Meter Table) :

- **Group Table** : Utilisée pour des tâches avancées comme **répliquer** un paquet vers plusieurs ports ou effectuer un **équilibrage de charge** entre plusieurs chemins.

- **Meter Table** : Contrôle le débit du trafic pour éviter la surcharge du réseau. Par exemple, elle peut limiter le nombre de paquets envoyés par seconde pour un flux donné.

6. Transmission du paquet :

Une fois le paquet traité (via une règle existante ou une instruction du contrôleur), le commutateur **transmet le paquet** vers le port de sortie approprié.

Commutateur OpenFlow (OpenFlow Switch) : Structure

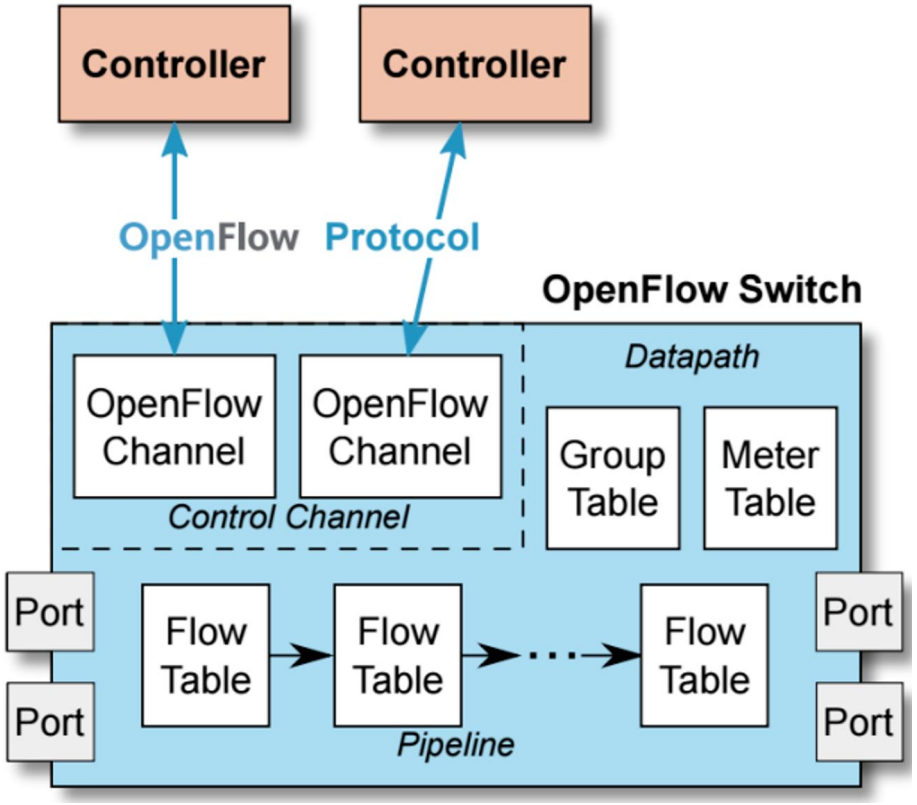


Schéma global du fonctionnement :

1. Un paquet arrive au commutateur via un port.
2. Le commutateur cherche une règle correspondante dans ses tables de flux :
    1. Si une correspondance est trouvée, il exécute l'action et envoie le paquet.
    2. Sinon, il consulte le contrôleur via OpenFlow.
3. Le contrôleur peut décider immédiatement ou installer une nouvelle règle.
4. Le commutateur met à jour ses tables de flux si nécessaire.
5. Le paquet est traité et envoyé au bon endroit via un port de sortie.

Pourquoi cette approche est efficace ?

- **Contrôle centralisé** : Le contrôleur SDN gère les décisions réseau de manière intelligente et centralisée.
- **Flexibilité** : Les règles peuvent être modifiées en temps réel par le contrôleur.
- **Optimisation** : Les commutateurs n'ont pas besoin de prendre des décisions complexes, ils suivent simplement les instructions du contrôleur.
- **Scalabilité** : En stockant localement les règles dans les Flow Tables, les commutateurs peuvent traiter les paquets rapidement sans surcharge du contrôleur.

OpenFlow Switch

Fonctionnement détaillé du Protocole

•**Switch OpenFlow** : Gère les **flow tables** contenant des règles sur la façon de traiter les paquets.
•**Flow Entry** (Entrée de flux) : Une règle contient plusieurs champs :

- **Match fields** : Adresse IP, port, protocole, etc.
- **Actions** : Transférer, modifier ou abandonner un paquet.
- **Counters** : Statistiques sur l'utilisation de chaque règle.

Processus typique :

1. **Réception du paquet** : Le switch compare les champs du paquet avec les règles de la table de flux.
2. **Absence de correspondance** : Le switch envoie le paquet au **contrôleur SDN** via un **message Packet-in**.
3. **Décision du contrôleur** : Le contrôleur décide de la règle à appliquer et envoie une instruction au switch via un **message Flow-mod**.

Messages d'OpenFlow

- **Packet-in** : Le switch informe le contrôleur d'un paquet inconnu.
- **Flow-mod** : Le contrôleur installe ou modifie une règle de flux sur le switch.
- **Packet-out** : Le contrôleur ordonne au switch d'envoyer un paquet spécifique sur une interface.
- **Stats Request / Reply** : Collecte de statistiques réseau du switch.

Questions & Réponses ?

Comment fonctionne un commutateur OpenFlow ?

Un commutateur OpenFlow ne peut fonctionner qu'avec la collaboration de trois éléments essentiels : les tables de flux installées sur les commutateurs, un contrôleur, et un protocole OpenFlow propriétaire permettant au contrôleur de communiquer de manière sécurisée avec les commutateurs.

- Les tables de flux sont configurées sur les commutateurs.
- Les contrôleurs communiquent avec les commutateurs via le protocole OpenFlow et imposent des politiques sur les flux.

Le contrôleur peut configurer des chemins à travers le réseau optimisé pour des caractéristiques spécifiques, telles que la vitesse, le nombre minimum de sauts ou la réduction de la latence.

Commutateur OpenFlow vs Commutateur Conventionnel : Quelle est la différence ?

- Dans un commutateur conventionnel, l'acheminement des paquets (le plan de données) et le routage de haut niveau (le plan de contrôle) se produisent sur le même dispositif.

- Alors que pour un commutateur OpenFlow, le plan de données est découplé du plan de contrôle :
  - le plan de données est implémenté dans le commutateur lui-même, tandis que le plan de contrôle est géré par un logiciel, et un contrôleur SDN distinct prend les décisions de routage de haut niveau.
  - Le commutateur et le contrôleur communiquent via le protocole OpenFlow.

Le commutateur OpenFlow offre des avantages :

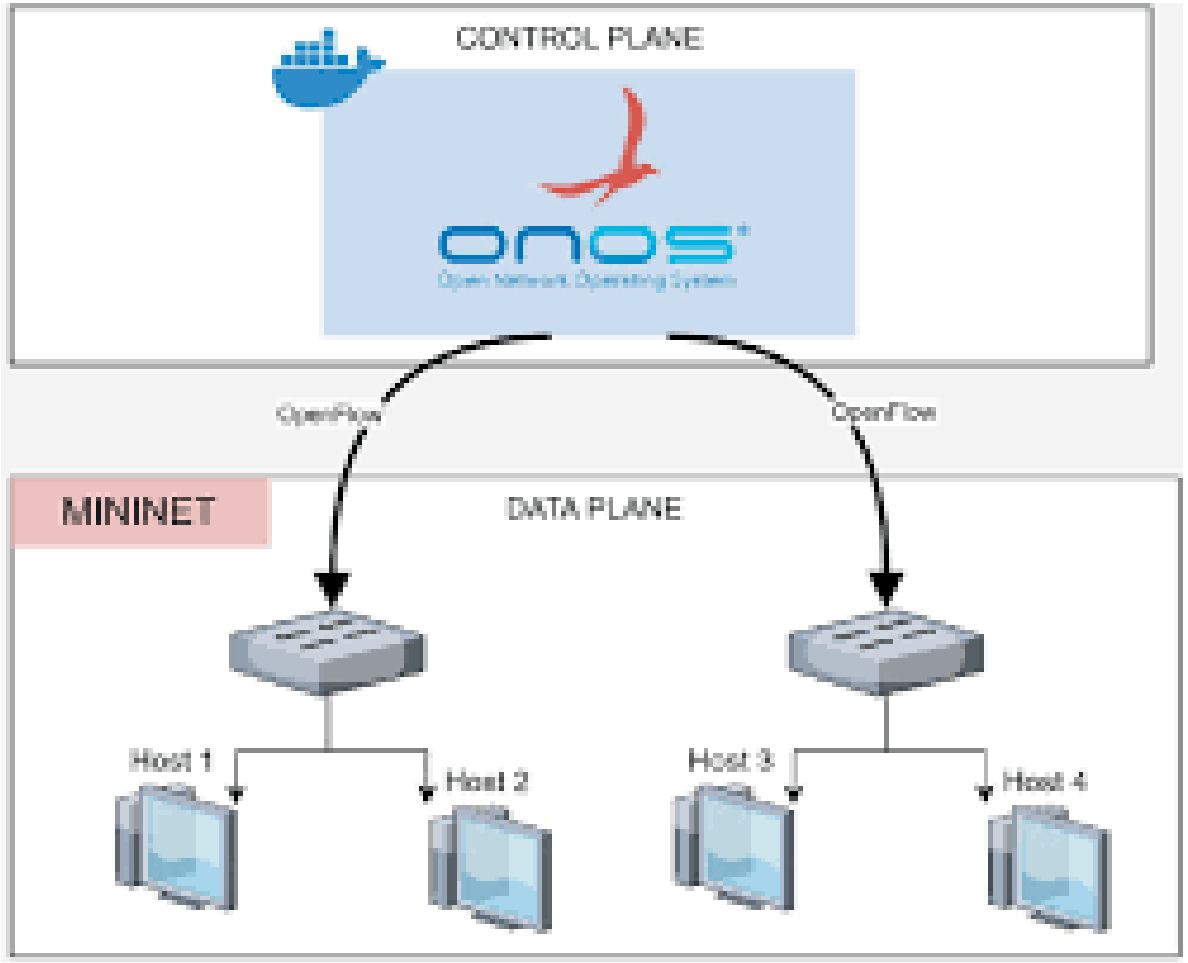
- Le contrôleur SDN peut facilement implémenter l'équilibrage de charge à des débits élevés en dirigeant simplement différents flux vers différents hôtes, en ne faisant que la configuration du flux initial.

- Le trafic peut être isolé sans avoir besoin de VLANs, car le contrôleur SDN du commutateur OpenFlow peut simplement refuser certaines connexions.

- Il est également possible de configurer un sniffer pour n'importe quel port, ou même pour un trafic spécifique, en programmant le réseau pour envoyer un flux dupliqué à un dispositif de surveillance réseau.

- OpenFlow permet également le développement de nouveaux services et idées entièrement par logiciel sur le contrôleur SDN, tout en accélérant la mise en place de nouvelles fonctionnalités et services.

RELATION: ONOS, OPENFLOW, MININET, OVS



Le rôle d'ONOS (Open Network Operating System)

A. Présentation d'ONOS

- **ONOS** est un **contrôleur SDN open-source** destiné aux grandes infrastructures (data centers, ISP, etc.).
- Il fournit une **interface centralisée** pour configurer et surveiller les équipements réseau en utilisant des protocoles comme **OpenFlow**, **NETCONF**, et **P4**.

B. Fonctionnalités clés d'ONOS

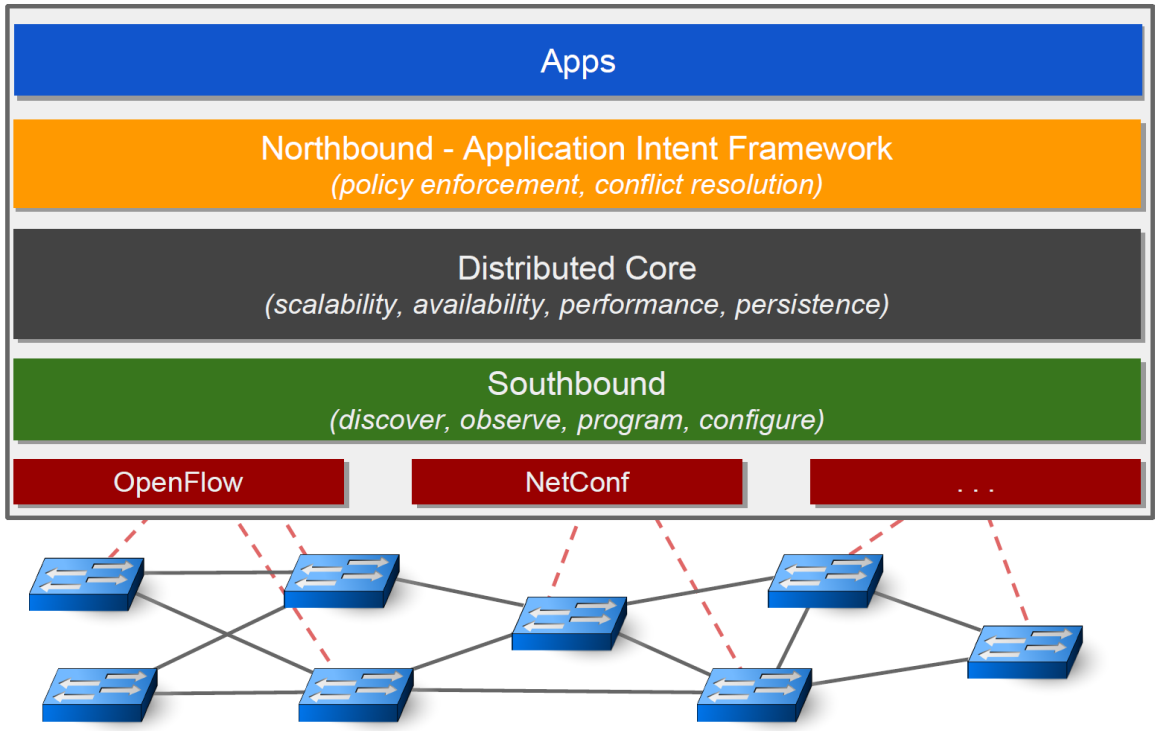
1. **Gestion centralisée du réseau** : Permet de gérer de nombreux commutateurs simultanément.
2. **High Availability (HA)** : Résilience avec plusieurs nœuds ONOS fonctionnant en parallèle.
3. **Support d'OpenFlow** : Installe et gère dynamiquement les flux à travers les switches compatibles.
4. **Programmabilité** : Fournit une API RESTful et des SDK (Java, Python) pour écrire des applications SDN.



Northbound Abstraction:
- network graph
- application intents

Core:
- distributed
- protocol independent

Southbound Abstraction:
- generalized OpenFlow
- pluggable & extensible



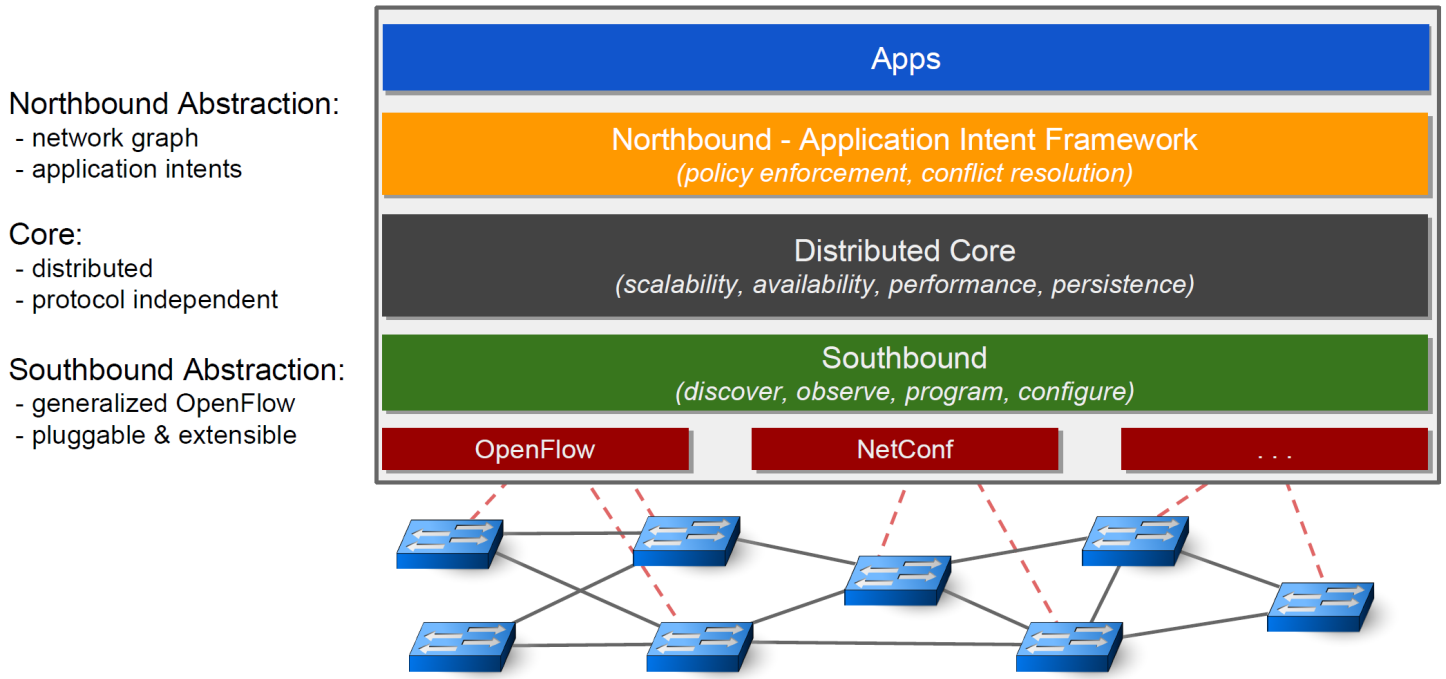
Relation entre OpenFlow, OVS, et ONOS

1. **OVS agit comme un datapath programmable** qui exécute les règles reçues via OpenFlow.

2. **ONOS est le cerveau du réseau** : Il envoie des règles à OVS en utilisant le protocole OpenFlow. Par exemple, ONOS peut installer des règles pour :

    1. **Équilibrer la charge** entre plusieurs chemins.
    2. **Isoler des flux** en créant des VLANs ou tunnels dynamiques.
    3. **Détecter et prévenir les intrusions** en bloquant certains types de trafic.

3. **L'interaction entre ONOS et OVS via OpenFlow** permet une gestion flexible, automatisée et centralisée du réseau.

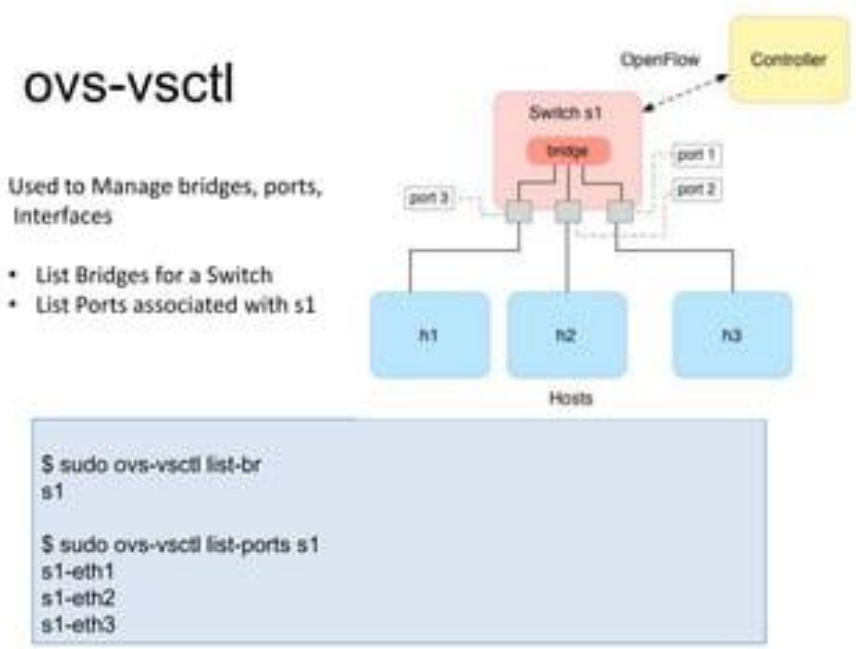


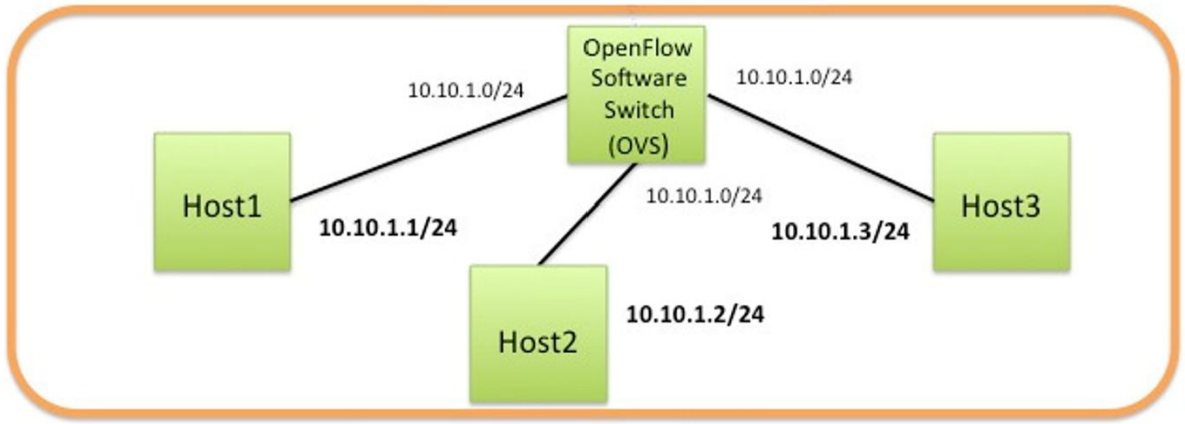
Open vSwitch (OVS) : Switch compatible OpenFlow

A. Qu'est-ce qu'Open vSwitch ?

- **Open vSwitch (OVS)** est un **commutateur virtuel** qui permet de gérer les flux à travers OpenFlow.
- Il est souvent utilisé dans des environnements **SDN**, **NFV** et **cloud** pour simuler ou virtualiser des réseaux.
- Compatible avec **OpenFlow**, OVS peut fonctionner comme un **datapath programmable**, capable de suivre les instructions envoyées par un contrôleur SDN.

Un **datapath programmable** fait référence à une architecture réseau où les éléments du chemin de données (datapath) peuvent être configurés ou modifiés dynamiquement par le logiciel





Fonctionnalités d'OVS

- **Support OpenFlow** : Fonctionne avec différents contrôleurs SDN (ONOS, OpenDaylight, etc.).
- **Routage programmable** : Dynamique et basé sur les règles reçues via OpenFlow.
- **Gestion VLAN et tunneling** : Gère les VLANs, GRE, et VXLAN pour la segmentation réseau.
- **Performance et compatibilité** : Peut être déployé dans des environnements Linux et cloud, et est utilisé dans des plateformes comme **OpenStack**.

Démonstration Pratique avec ONOS, OVS et OpenFlow: Exemple

Lancement de ONOS
Visualisation sous ONOS



Création de règles de flux
Et applications

Création d'une topologie avec mininet

Démonstration Pratique avec ONOS, OVS et OpenFlow: Exemple

